Building a Scalable Agentic LLM Serving System.

Alis

https://softmaxxer101.github.io/
https://x.com/import_alis

Date: 11th November 2026

Abstract- This technical abstract outlines the a reproducible framework for serving agentic workloads with Qwen3-0.6B through vLLM on constrained GPUs. It builds deterministic traces, benchmarking harnesses, telemetry, experiments, and failure tests to measure latency, throughput, caching, scheduling, memory utilization, scalability, and cost while establishing a foundation for scaling to larger models and improved hardware. The harness used throughout this experiment is a custom Python benchmarking harness built around vLLM's OpenAI-compatible API.

Index Terms- Agentic AI, LLM serving, vLLM, Qwen3, GPU inference, benchmarking, latency, throughput, KV cache, scalability, observability, cost optimization.

I. THE HARDWARE SETUP

WE will create an env and put variables used throughout the program inside it:
Put in envvars.sh and source it:

```
export GPU=3050
export KEY=lab-secret-key
export MODEL=Qwen/Qwen3-0.6B
export HF_HOME=$HOME/hf
export HF_HUB_ENABLE_HF_TRANSFER=1
export PYTHONPATH=$PWD/harness
```

bash : #source env/vars.sh everytime you restart your env

If you have an existing env bash the below:

#pip install openai numpy pandas matplotlib uvloop transformers hf_transfer fastapi uvicorn httpx
python -c 'import vllm,torch;print(vllm.__version__,torch.__version__,torch.version.cuda)' | tee env/versions.txt
pip freeze > env/pip-freeze.txt

Download the model – Qwen3-0.6B

#mkdir -p $HF_HOME
hf download $MODEL # older versions: huggingface-cli download $MODEL
du -sh $HF_HOME
cat $HF_HOME/hub/models--Qwen--Qwen3-0.6B/snapshots/*/config.json | tee env/model-config.json

Start the server:

#vllm serve $MODEL --max-model-len 8192 --gpu-memory-utilization 0.75 --max-num-seqs 32 --reasoning-parser qwen3 --enable-auto-tool-choice --tool-call-parser hermes --enable-prompt-tokens-details --enforce-eager --api-key $KEY 2>&1 | tee runs/smoke-server.log

After the server start up extract the facts:

#grep -iE 'kv cache|maximum concurrency|prefix|chunked|cuda graph|enforce|backend|quantization|dtype|max_model_len|model loading took|memory' runs/smoke-server.log | tee env/startup-facts.txt

Save the below as harness/smoke.py:

```
#from openai import OpenAI
import os
c = OpenAI(base_url='http://localhost:8000/v1', api_key=os.environ['KEY'])
m = 'Qwen/Qwen3-0.6B'
def rs(msg): return getattr(msg, 'reasoning_content', None) or getattr(msg, 'reasoning', None)
r = c.chat.completions.create(model=m, messages=[{'role': 'user', 'content': 'Say hi in 5 words.'}], max_tokens=400)
print('PLAIN content:', r.choices[0].message.content); print('PLAIN reasoning:', rs(r.choices[0].message)); print(r.usage)
r = c.chat.completions.create(model=m, messages=[{'role': 'user', 'content': 'Say hi in 5 words.'}], max_tokens=100, extra_body={'chat_template_kwargs': {'enable_thinking': False}})
print('NOTHINK content:', r.choices[0].message.content); print(r.usage)
tools = [{'type': 'function', 'function': {'name': 'calculator', 'description': 'Evaluate math', 'parameters': {'type': 'object', 'properties': {'expr': {'type': 'string'}}, 'required': ['expr']}}}]
r = c.chat.completions.create(model=m, messages=[{'role': 'user', 'content': 'What is 123*45? Use the tool.'}], tools=tools, max_tokens=500)
print('TOOLS:', r.choices[0].message.tool_calls, r.choices[0].finish_reason)
```

run: python harness/smoke.py | tee runs/smoke-test-output.txt

```
(venv) ali5@LAPTOP-N3QNJPKH:~/project/agentic-serving-lab$ python harness/smoke.py | tee runs/s
moke-test-output.txt
PLAIN content:

Hi there!
PLAIN reasoning:
Okay, the user wants me to say "Hi" in five words. Let me think. First, I need to make sure it'
s exactly five words. "Hi there!" is four words. So I need to add one more. Maybe "Hi, there!"
That's five words. But wait, "Hi, there!" is still four. Hmm. Alternatively, "Hi, there!" or "H
i, there!" Still four. Wait, maybe "Hi, there!" is correct. Let me count again: H-I-T-H, then t
here? No, that's three. Wait, no, the user said "in five words." So maybe "Hi, there!" is four.
 Maybe the user wants to include a comma? Like "Hi, there!" is four. Wait, maybe the user made
a mistake. But I need to respond in five words. Let me think again. "Hi, there!" is four. So ma
ybe "Hi, there!" is acceptable. Alternatively, "Hi, there!" is correct. Wait, maybe the user wa
nts to include a space. "Hi there!" is five words. Yes, that's correct. So the answer is "Hi th
ere!".

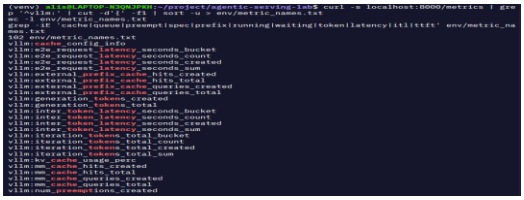
CompletionUsage(completion_tokens=252, prompt_tokens=15, total_tokens=267, completion_tokens_de
tails=CompletionTokensDetails(accepted_prediction_tokens=None, audio_tokens=None, reasoning_tok
ens=245, rejected_prediction_tokens=None, text_tokens=None), prompt_tokens_details=PromptTokens
Details(audio_tokens=None, cache_write_tokens=None, cached_tokens=0, image_tokens=None, text_to
kens=None, created_cache_tokens=0, multimodal_tokens=None))
NOTHINK content: Hello! How can I assist you today?
CompletionUsage(completion_tokens=10, prompt_tokens=19, total_tokens=29, completion_tokens_deta
ils=CompletionTokensDetails(accepted_prediction_tokens=None, audio_tokens=None, reasoning_token
s=0, rejected_prediction_tokens=None, text_tokens=None), prompt_tokens_details=PromptTokensDeta
ils(audio_tokens=None, cache_write_tokens=None, cached_tokens=16, image_tokens=None, text_token
s=None, created_cache_tokens=0, multimodal_tokens=None))
TOOLS: [ChatCompletionMessageFunctionToolCall(id='chatcmpl-tool-96ea245509e98021', function=Fun
ction(arguments='{"expr": "123*45"}', name='calculator'), type='function')] tool_calls
```

Checklist:

The chat return text output: Hi there!
Reasoning appeared separately
Non-thinking mode returns output with zero reasoning and only text produced.
Tool calling worked and the calculator is working, the computation {"expr": "123*45"} ran successfully.
Prompt token details are available.

Save the metrics name:

```
# curl -s localhost:8000/metrics | grep '^vllm:' | cut -d'{' -f1 | sort -u > env/metric_names.txt
wc -l env/metric_names.txt
grep -iE 'cache|queue|preempt|spec|prefix|running|waiting|token|latency|itl|ttft' env/metric_names.txt
```



→ a huge ouput like this would appear

Important metrics for this project are given below:

- Requests: vllm:num_requests_running, vllm:num_requests_waiting
- KV cache: vllm:kv_cache_usage_perc
- Preemption: vllm:num_preemptions_total
- Prefix caching: vllm:prefix_cache_hits_total, vllm:prefix_cache_queries_total
- Prompt caching: vllm:prompt_tokens_cached_total
- Throughput: vllm:generation_tokens_total
- TTFT: vllm:time_to_first_token_seconds_*
- Inter-token latency: vllm:inter_token_latency_seconds_*
- End-to-end latency: vllm:e2e_request_latency_seconds_*
- Queue time: vllm:request_queue_time_seconds_*

Stop the server : execute Ctrl + C before entering the next phase

Phase 1:

D=$(python -c 'import os,json;print(os.path.dirname(json.__file__))')
cat $(ls $D/*.py $(dirname $D)/*.py | head -200) > traces/corpus.txt
wc -c traces/corpus.txt

```
(venv) alis@LAPTOP-N3QNJPKH:~/project/agentic-serving-lab$ D=$(python -c 'import os,json;print(
os.path.dirname(json.__file__))')
cat $(ls $D/*.py $(dirname $D)/*.py | head -200) > traces/corpus.txt
wc -c traces/corpus.txt
4674931 traces/corpus.txt
```

Expected output: few million characters should be returned. The output showed 4674931, this is enough space for the harness. Now record its SHA-256 hash:

sha256sum traces/corpus.txt | tee env/corpus.sha

The corpus is a fake tool, not what real production systems use. This is being used in this project because the project's first aim is to measure the serving system, not the quality or speed of external tools.

Step 1.2 : shared code in **harness/common.py**

```
# import time, json
from openai import AsyncOpenAI
NAMES = ['search_docs', 'read_file', 'run_python', 'sql_query', 'calculator', 'final_answer']
TOOLS_SPEC = [{'type': 'function', 'function': {'name': n, 'description': 'Mock tool ' + n, 'parameters': {'type': 'object', 'properties': {'query': {'type': 'string'}}, 'required': ['query']}}} for n in NAMES]
_client = None
def client(a):
    global _client
    if _client is None: _client = AsyncOpenAI(base_url=a.base_url, api_key=a.api_key, timeout=3600)
    return _client
async def call(a, messages, tools, think):
    c = client(a)
    kw = {} if think else {'enable_thinking': False}
    samp = dict(temperature=0.6, top_p=0.95) if think else dict(temperature=0.7, top_p=0.8)
    args = dict(model=a.model, messages=messages, stream=True, stream_options={'include_usage': True}, max_tokens=a.max_tokens, extra_body={'top_k': 20, 'chat_template_kwargs': kw}, **samp)
    if tools: args['tools'] = tools
    ts = time.time(); t0 = time.perf_counter(); first = answer = None; last = t0
    gaps = []; content = ''; reasoning = ''; calls = {}; usage = None; finish = None
    stream = await c.chat.completions.create(**args)
    async for ch in stream:
        now = time.perf_counter()
        if ch.usage: usage = ch.usage
        if not ch.choices: continue
        d = ch.choices[0].delta
        if ch.choices[0].finish_reason: finish = ch.choices[0].finish_reason
        think_t = getattr(d, 'reasoning_content', None) or getattr(d, 'reasoning', None)
        if think_t or d.content or d.tool_calls:
            if first is None: first = now - t0
            else: gaps.append(now - last)
            last = now
```

```
            if think_t: reasoning += think_t
            if d.content:
                content += d.content
                if answer is None: answer = now - t0
            for tc in d.tool_calls or []:
                if answer is None: answer = now - t0
                s = calls.setdefault(tc.index, {'id': tc.id, 'name': '', 'args': ''})
                if tc.id: s['id'] = tc.id
                if tc.function and tc.function.name: s['name'] += tc.function.name
                if tc.function and tc.function.arguments: s['args'] += tc.function.arguments
    e2e = time.perf_counter() - t0
    n = usage.completion_tokens if usage else None
    ok = True
    for s in calls.values():
        try: json.loads(s['args'] or '{}')
        except Exception: ok = False
    det = getattr(usage, 'prompt_tokens_details', None) if usage else None
    cached = getattr(det, 'cached_tokens', None) if det else None
    return dict(t_submit=ts, ttft=first, t_answer=answer, e2e=e2e, finish=finish, prompt_tokens=usage.prompt_tokens if usage else None, completion_tokens=n, cached_tokens=cached,
reasoning_chars=len(reasoning), mean_itl=((e2e - first) / max(n - 1, 1)) if first and n else None, chunk_gaps=gaps, tool_calls=list(calls.values()), tool_json_ok=ok, content=content)
```

the sampling settings used by the harness.

For thinking requests:
temperature = 0.6
top_p = 0.95
top_k = 20
For non-thinking requests:
temperature = 0.7
top_p = 0.8
top_k = 20

These settings allow controlled randomness. Greedy-algorithm is discouraged for this model. It can make the model repetitive and unnatural.

Step 1.3: Trace generator

#vim harness/gen_trace.py

```
# import json, math, random, hashlib, argparse
from transformers import AutoTokenizer
ap = argparse.ArgumentParser()
ap.add_argument('--n', type=int, default=300); ap.add_argument('--seed', type=int, default=1)
ap.add_argument('--projects', type=int, default=4); ap.add_argument('--scale', type=float, default=1.0)
ap.add_argument('--mix', default='A:0.4,B:0.2,C:0.3,D:0.1'); ap.add_argument('--out', default='traces/M.jsonl')
a = ap.parse_args()
tok = AutoTokenizer.from_pretrained('Qwen/Qwen3-0.6B')
ids = tok(open('traces/corpus.txt').read()[:3000000])['input_ids']
rng = random.Random(a.seed)
def text(n, seed):
    r = random.Random(seed); s = r.randrange(0, len(ids) - n - 1); return tok.decode(ids[s:s + n])
def sha(x): return int(hashlib.sha256(x.encode()).hexdigest()[:8], 16)
def lognorm(med, p95):
    sig = (math.log(p95) - math.log(med)) / 1.645
    return max(1, int(med * math.exp(rng.gauss(0, sig))))
P = {'A': dict(steps=(6, 15), tool=(300, 1200), think=True, delay=(0.3, 2.0)),
     'B': dict(steps=(5, 12), tool=(600, 2000), think=True, delay=(0.5, 3.0)),
     'C': dict(steps=(3, 6), tool=(100, 300), think=False, delay=(0.1, 0.5)),
     'D': dict(steps=(1, 1), tool=None, think=False, delay=None)}
TOOLS = ['search_docs', 'read_file', 'run_python', 'sql_query', 'calculator']
SYS = text(int(600 * a.scale), 999)
PROJ = {p: text(int(rng.randint(2000, 3000) * a.scale), 5000 + p) for p in range(a.projects)}
cl, w = zip(*[(c, float(x)) for c, x in (kv.split(':') for kv in a.mix.split(','))])
out = open(a.out, 'w')
for i in range(a.n):
    c = rng.choices(cl, w)[0]; p = P[c]
    s = dict(id='s%d' % i, cls=c, gap=rng.expovariate(1.0), think=p['think'], script=[], answer='', system='' if c == 'D' else SYS, ctx='')
    if c == 'A': s['project'] = rng.randrange(a.projects); s['ctx'] = PROJ[s['project']]
    if c == 'D':
        s['ctx'] = text(int(rng.randint(3000, 6000) * a.scale), 7000 + i); s['task'] = 'Summarize the document in 3 sentences.'; s['max_steps'] = 1
    else:
        nsteps = max(1, min(p['steps'][1], round(rng.gauss(p['steps'][0], p['steps'][0] / 3))))
        val = str(sha('v%d' % i)); key = 'KEY_%d' % i; hide = rng.randrange(nsteps) if nsteps > 0 else 0
        s['answer'] = val; s['task'] = 'Use the tools to find %s in the tool outputs, then reply exactly: Answer: <value>.' % key
        for k in range(nsteps):
            name = rng.choice(TOOLS); args = json.dumps({'query': '%s-%d' % (s['id'], k)})
            body = text(lognorm(*p['tool']), sha(name + args))
            if k == hide: body = '%s = %s ' % (key, val) + body
            tid = 'call_%s_%d' % (s['id'], k)
            s['script'].append(dict(assistant={'role': 'assistant', 'content': '', 'tool_calls': [{'id': tid, 'type': 'function', 'function': {'name': name, 'arguments': args}}]}, tool_results=[{'role': 'tool', 'tool_call_id': tid, 'content': body}],
delay=min(lognorm(*[int(x * 1000) for x in p['delay']]) / 1000, 20)))
        s['script'].append(dict(assistant={'role': 'assistant', 'content': 'Answer: ' + val}, tool_results=[], delay=0))
        s['max_steps'] = len(s['script'])
    out.write(json.dumps(s) + chr(10))
```

```
(venv) alis@LAPTOP-N3QNJPKH:~/project/agentic-serving-lab$ python harness/gen_trace.py --n 300
--seed 1 --projects 4 --out traces/M.jsonl
python harness/gen_trace.py --n 300 --seed 1 --projects 4 --out traces/M2.jsonl
sha256sum traces/M.jsonl traces/M2.jsonl | tee env/trace-hashes.txt
/home/alis/project/venv/lib/python3.12/site-packages/huggingface_hub/constants.py:302: FutureWa
rning: The `HF_HUB_ENABLE_HF_TRANSFER` environment variable is deprecated as 'hf_transfer' is n
ot used anymore. Please use `HF_XET_HIGH_PERFORMANCE` instead to enable high performance transf
er with Xet. Visit https://huggingface.co/docs/huggingface_hub/package_reference/environment_va
riables#hfxethighperformance for more details.
  warnings.warn(
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enabl
e higher rate limits and faster downloads.
[transformers] Token indices sequence length is longer than the specified maximum sequence leng
th for this model (710070 > 131072). Running this sequence through the model will result in ind
exing errors
/home/alis/project/venv/lib/python3.12/site-packages/huggingface_hub/constants.py:302: FutureWa
rning: The `HF_HUB_ENABLE_HF_TRANSFER` environment variable is deprecated as 'hf_transfer' is n
ot used anymore. Please use `HF_XET_HIGH_PERFORMANCE` instead to enable high performance transf
er with Xet. Visit https://huggingface.co/docs/huggingface_hub/package_reference/environment_va
riables#hfxethighperformance for more details.
  warnings.warn(
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enabl
e higher rate limits and faster downloads.
[transformers] Token indices sequence length is longer than the specified maximum sequence leng
th for this model (710070 > 131072). Running this sequence through the model will result in ind
exing errors
956b8631cc401da9f991c2edbb5d933832e36eecd46c3ab67dddf0adcd3e46ec  traces/M.jsonl
956b8631cc401da9f991c2edbb5d933832e36eecd46c3ab67dddf0adcd3e46ec  traces/M2.jsonl
```

The two trace files generated same hashes as expected, this confirms deterministic trace generation.

python harness/gen_trace.py --n 300 --seed 1 --mix A:1 --out traces/A.jsonl
python harness/gen_trace.py --n 300 --seed 1 --mix B:1 --out traces/B.jsonl
python harness/gen_trace.py --n 300 --seed 1 --mix C:1 --out traces/C.jsonl
python harness/gen_trace.py --n 300 --seed 1 --mix D:1 --out traces/D.jsonl

All 4 classes-traces were generated successfully.

Step 1.4: **Trace checker: harness/check_trace.py**

```
# import json, sys, copy
from transformers import AutoTokenizer
from common import TOOLS_SPEC
tok = AutoTokenizer.from_pretrained('Qwen/Qwen3-0.6B'); LIM = int(sys.argv[2]); B = 16
seen = set(); tot = hit = over = mx = nreq = 0
for l in open(sys.argv[1]):
    s = json.loads(l); msgs = []
    if s['system']: msgs.append({'role': 'system', 'content': s['system']})
    if s['ctx']: msgs.append({'role': 'user', 'content': 'Context: ' + s['ctx']})
    msgs.append({'role': 'user', 'content': s['task']})
    for k in range(max(1, len(s['script']))):
        text = tok.apply_chat_template(msgs, tools=TOOLS_SPEC, add_generation_prompt=True, tokenize=False)
        ids = tok(text)['input_ids']; n = len(ids); mx = max(mx, n); over += n + 512 > LIM; nreq += 1
        h = 0; run = True; nb = n // B
        for b in range(nb):
            h = hash((h, tuple(ids[b * B:(b + 1) * B])))
            if run and h in seen: hit += 1
            else: run = False
            seen.add(h)
        tot += nb
        if k < len(s['script']):
            st = copy.deepcopy(s['script'][k])
            for t in st['assistant'].get('tool_calls', []): t['function']['arguments'] = json.loads(t['function']['arguments'])
            msgs.append(st['assistant']); msgs.extend(st['tool_results'])
print('requests', nreq, 'max prompt tokens', mx, 'over limit (prompt+512 > LIM)', over, 'ideal hit rate %.3f' % (hit / tot))
```

Run: python harness/check_trace.py traces/M.jsonl 8192 | tee env/trace-check-M.txt

python harness/gen_trace.py \
  --n 300 --seed 1 --projects 8 --scale 0.20 \
  --out traces/M-p8.jsonl

python harness/gen_trace.py \
  --n 300 --seed 1 --projects 16 --scale 0.20 \
  --out traces/M-p16.jsonl

python harness/check_trace.py traces/M-p8.jsonl 8192 \
  | tee env/trace-check-M-p8.txt

python harness/check_trace.py traces/M-p16.jsonl 8192 \
  | tee env/trace-check-M-p16.txt

Trace	Projects	Max prompt	Over limit	Ideal cache hit rate
M	4	11,657	61	87.5%
M-p8	8	9,670	18	81.3%
M-p16	16	9,670	14	81.2%

Step 1.5 Session runner: harness/run.py

```
# import asyncio, json, time, argparse, uvloop
from common import call, TOOLS_SPEC
ap = argparse.ArgumentParser()
ap.add_argument('--trace', required=True); ap.add_argument('--mode', default='replay', choices=['replay', 'live'])
ap.add_argument('--load', default='closed', choices=['open', 'closed']); ap.add_argument('--rate', type=float, default=0.5)
ap.add_argument('--conc', type=int, default=4); ap.add_argument('--n', type=int, default=100); ap.add_argument('--warmup', type=int, default=8)
ap.add_argument('--max-tokens', dest='max_tokens', type=int, default=512)
ap.add_argument('--base-url', dest='base_url', default='http://localhost:8000/v1'); ap.add_argument('--api-key', dest='api_key', default='lab-secret-key')
ap.add_argument('--model', default='Qwen/Qwen3-0.6B'); ap.add_argument('--out', required=True)
a = ap.parse_args()
def prefix(s):
    m = []
    if s['system']: m.append({'role': 'system', 'content': s['system']})
    if s['ctx']: m.append({'role': 'user', 'content': 'Context: ' + s['ctx']})
    m.append({'role': 'user', 'content': s['task']}); return m
async def run_session(s, sink):
    msgs = prefix(s); t_start = time.time(); steps = 0; ok = False
    tools = TOOLS_SPEC if s['cls'] != 'D' else None
    for k in range(s['max_steps']):
        try: r = await call(a, msgs, tools, s['think'])
        except Exception as e:
            sink({'session': s['id'], 'step': k, 'cls': s['cls'], 'error': repr(e), 't_submit': time.time()}); break
        steps += 1
        sink({**{x: r[x] for x in r if x != 'content'}, 'session': s['id'], 'step': k, 'cls': s['cls']})
        if s['cls'] == 'D': break
        st = s['script'][min(k, len(s['script']) - 1)]
        if a.mode == 'replay':
            msgs.append(st['assistant'])
            if not st['assistant'].get('tool_calls'): break
        else:
            if not r['tool_calls']:
                ok = s['answer'] in r['content']; break
            msgs.append({'role': 'assistant', 'content': r['content'], 'tool_calls': [{'id': t['id'] or 'c%d' % k, 'type': 'function', 'function': {'name': t['name'], 'arguments': t['args']}} for t in r['tool_calls']]})
        await asyncio.sleep(st['delay'])
        if a.mode == 'replay': msgs.extend(st['tool_results'])
        else:
            for t in r['tool_calls']: msgs.append({'role': 'tool', 'tool_call_id': t['id'] or 'c%d' % k, 'content': st['tool_results'][0]['content'] if st['tool_results'] else 'ok'})
    sink({'session': s['id'], 'cls': s['cls'], 'session_done': True, 'steps': steps, 'dur': time.time() - t_start, 'success': ok if a.mode == 'live' else None, 't_start': t_start})
async def go(ss, sink, rate=None, conc=4):
    if rate:
        ts = []
        for s in ss:
            await asyncio.sleep(s['gap'] / rate); ts.append(asyncio.create_task(run_session(s, sink)))
        await asyncio.gather(*ts)
    else:
        it = iter(ss)
        async def worker():
            for s in it: await run_session(s, sink)
        await asyncio.gather(*[worker() for _ in range(conc)])
async def main():
    allss = [json.loads(l) for l in open(a.trace)]
    warm, ss = allss[:a.warmup], allss[a.warmup:a.warmup + a.n]
    f = open(a.out, 'a')
    def sink(r): f.write(json.dumps(r) + chr(10)); f.flush()
    if warm: await go(warm, lambda r: None, None, 4)
    sink({'run_start': time.time(), 'args': vars(a)})
    await go(ss, sink, a.rate if a.load == 'open' else None, a.conc)
    sink({'run_end': time.time()})
uvloop.run(main())
```

create this file and store it for later reference.

Step 1.6 Server-side metrics scraper: harness/scrape.py

#vim harness/scrape.py

Start the server

```
#import time, sys, urllib.request, csv, re
url, out = sys.argv[1], sys.argv[2]
w = csv.writer(open(out, 'w', buffering=1)); w.writerow(['ts', 'name', 'labels', 'value'])
while True:
    try: txt = urllib.request.urlopen(url, timeout=2).read().decode()
    except Exception:
        time.sleep(1); continue
    ts = time.time()
    for l in txt.splitlines():
        if l.startswith('vllm:') and '_bucket' not in l:
            m = re.match(r'([^{ ]+)(\{.*\})? (\S+)$', l)
            if m: w.writerow([ts, m.group(1), m.group(2) or '', m.group(3)])
    time.sleep(1)
```

Step 1.7 Mock server (test the harness without the GPU)

Save as mock_server/app.py:

```
#from fastapi import FastAPI, Request
from fastapi.responses import StreamingResponse, PlainTextResponse
import asyncio, json
app = FastAPI()
@app.get('/health')
async def health(): return PlainTextResponse('ok')
@app.get('/metrics')
async def metrics(): return PlainTextResponse('vllm:num_requests_running 1' + chr(10))
@app.post('/v1/chat/completions')
async def chat(req: Request):
    body = await req.json()
    async def gen():
        for i in range(20):
            await asyncio.sleep(0.01)
            yield 'data: ' + json.dumps({'id': 'x', 'object': 'chat.completion.chunk', 'model': 'm', 'choices': [{'index': 0, 'delta': {'content': 'tok '}, 'finish_reason': None}]}) + chr(10) * 2
        yield 'data: ' + json.dumps({'id': 'x', 'object': 'chat.completion.chunk', 'model': 'm', 'choices': [{'index': 0, 'delta': {}, 'finish_reason': 'stop'}]}) + chr(10) * 2
        yield 'data: ' + json.dumps({'id': 'x', 'object': 'chat.completion.chunk', 'model': 'm', 'choices': [], 'usage': {'prompt_tokens': 100, 'completion_tokens': 20, 'total_tokens': 120}}) + chr(10) * 2
        yield 'data: [DONE]' + chr(10) * 2
    return StreamingResponse(gen(), media_type='text/event-stream')
```

Test:

```
uvicorn mock_server.app:app --port 8000 &
python harness/run.py --trace traces/M.jsonl --mode replay --load closed --conc 8 --n 30 --warmup 2 --out /tmp/mock.jsonl
python harness/run.py --trace traces/M.jsonl --mode replay --load open --rate 5 --n 30 --warmup 2 --out /tmp/mock-open.jsonl
kill %1; wc -l /tmp/mock*.jsonl
```

Step 1.8 Config runner: run_config.sh

Each experiment is a file configs/<name>.env that defines EXP, SERVE_ARGS, WL_ARGS and optionally ENVV. Example configs/e0-base.env (3050):

```
EXP=e0
SERVE_ARGS='--max-model-len 8192 --gpu-memory-utilization 0.85 --max-num-seqs 32 --reasoning-parser qwen3 --enable-auto-tool-choice --tool-call-parser hermes --enable-prompt-tokens-details'
WL_ARGS='--trace traces/M.jsonl --mode replay --load closed --conc 4 --n 100'
ENVV=''
```

Save as run_config.sh and chmod +x

```
#!/usr/bin/env bash
set -uo pipefail
source env/vars.sh
CFG=$1; source configs/$CFG.env
OUT=runs/$GPU/$EXP/$CFG; mkdir -p $OUT; cp configs/$CFG.env $OUT/config.env
cleanup() { kill $P1 $P2 $P3 $SP 2>/dev/null; sleep 3; }
trap cleanup EXIT
nvidia-smi --query-gpu=timestamp,index,utilization.gpu,memory.used,power.draw,clocks.sm,temperature.gpu --format=csv -l 1 > $OUT/gpu.csv & P1=$!
nvidia-smi dmon -s pucvmet -d 1 > $OUT/dmon.csv & P2=$!
T0=$(date +%s)
env $ENVV vllm serve $MODEL $SERVE_ARGS --port 8000 --api-key $KEY > $OUT/server.log 2>&1 & SP=$!
for i in $(seq 1 400); do
  curl -sf localhost:8000/health > /dev/null && break
  kill -0 $SP 2>/dev/null || { echo 'server died, see server.log'; exit 1; }
  sleep 2
done
STARTUP=$(( $(date +%s) - T0 ))
python harness/scrape.py http://localhost:8000/metrics $OUT/metrics.csv & P3=$!
python - <<PYEOF > $OUT/meta.json
import json, vllm, torch, subprocess, hashlib
print(json.dumps(dict(vllm=vllm.__version__, torch=torch.__version__, cuda=torch.version.cuda, gpu='$GPU', startup_s=$STARTUP, serve_args='$SERVE_ARGS', envv='$ENVV', wl='$WL_ARGS',
trace_sha=hashlib.sha256(open('traces/M.jsonl','rb').read()).hexdigest())))
PYEOF
grep -iE 'kv cache size|maximum concurrency|model loading took' $OUT/server.log > $OUT/startup-facts.txt
python harness/run.py $WL_ARGS --out $OUT/requests.jsonl 2>&1 | tee $OUT/client.log
echo done $CFG
```

Run: ./run_config.sh e0-base

Step 1.9 Summary script: analysis/summarize.py

```
import json, sys, pandas as pd, numpy as np
rows = []
for path in sys.argv[1:]:
    recs = [json.loads(l) for l in open(path)]
    req = pd.DataFrame([r for r in recs if 'ttft' in r and 'session_done' not in r])
    ses = pd.DataFrame([r for r in recs if r.get('session_done')])
    err = sum(1 for r in recs if 'error' in r)
    st = [r for r in recs if 'run_start' in r][0]['run_start']; en = [r for r in recs if 'run_end' in r][0]['run_end']; dur = en - st
    q = lambda s, p: float(np.nanpercentile(s.dropna(), p)) if len(s.dropna()) else None
    rows.append(dict(run=path.split('/')[-2], sessions=len(ses), requests=len(req), errors=err, dur_s=round(dur, 1),
        ttft_p50=q(req.ttft, 50), ttft_p99=q(req.ttft, 99), itl_p50=q(req.mean_itl, 50), itl_p95=q(req.mean_itl, 95),
        e2e_p50=q(req.e2e, 50), e2e_p99=q(req.e2e, 99), sess_p50=q(ses.dur, 50), sess_p99=q(ses.dur, 99),
        out_tok_s=req.completion_tokens.sum() / dur, in_tok_s=req.prompt_tokens.sum() / dur,
        json_ok=req.tool_json_ok.mean(), cached_frac=(req.cached_tokens.sum() / req.prompt_tokens.sum()) if 'cached_tokens' in req and req.cached_tokens.notna().any() else None,
        truncated=(req.finish == 'length').mean()))
pd.set_option('display.width', 250); pd.set_option('display.max_columns', 50)
print(pd.DataFrame(rows).round(3).to_string(index=False))
```

Run: python analysis/summarize.py runs/$GPU/e0/e0-base/requests.jsonl

Step 1.10 Phase 1 gate

Phase 1b: Config tooling (do once, before experiments)

Step 1b.1 Patch run_config.sh

Five replacements so complex flags (JSON arguments), repeats, and clean shutdown all work.

1. Output folder per repeat. Replace the OUT line with:

```
OUT=runs/$GPU/$EXP/$CFG/rep${REP:-1}; mkdir -p $OUT; cp configs/$CFG.env $OUT/config.env
```

2. Start the server through eval so quoted JSON flags survive, and make the background PID the real server. Replace the vllm serve line with:
3. export SERVE_ARGS WL_ARGS ENVV GPU

```
( eval "exec env $ENVV vllm serve $MODEL $SERVE_ARGS --port 8000 --api-key $KEY" ) > $OUT/server.log 2>&1 & SP=$!
```

4. Make metadata read from the environment. Replace the python heredoc with (lines at column 0, and add export STARTUP before it):
5. python - > $OUT/meta.json <<'PYEOF'
6. import json, os, vllm, torch, hashlib
7. print(json.dumps(dict(vllm=vllm.__version__, torch=torch.__version__, cuda=torch.version.cuda, gpu=os.environ['GPU'], startup_s=int(os.environ['STARTUP']), serve_args=os.environ['SERVE_ARGS'], envv=os.environ['ENVV'], wl=os.environ['WL_ARGS'], trace_sha=hashlib.sha256(open('traces/M.jsonl','rb').read()).hexdigest()))) PYEOF
8. Make cleanup also remove stragglers and confirm VRAM is free. Replace the cleanup function with:

```
cleanup() { kill $P1 $P2 $P3 $SP 2>/dev/null; pkill -f 'vllm serve' 2>/dev/null; sleep 8; nvidia-smi --query-gpu=memory.used --format=csv,noheader >> $OUT/vram-after.txt; }
```

9. Optional quality probe after the workload. Add before the final echo:

```
[ -n "${PROBE:-}" ] && python harness/quality_probe.py $OUT/probe.json
```

In analysis/summarize.py, replace run=path.split('/')[-2] with run='/'.join(path.split('/')[-3:-1]) so repeats are labelled.

Observe: after every run the file vram-after.txt must show VRAM back near the idle value you recorded in Step 0.1. If not, a server is still alive: run nvidia-smi and kill it before the next run, or every later result is wrong.

Step 1b.2 Config generator: configs/mk.sh

```
BASE_3050='--max-model-len 8192 --gpu-memory-utilization 0.85 --max-num-seqs 32 --reasoning-parser qwen3 --enable-auto-tool-choice --tool-call-parser hermes --enable-prompt-tokens-details'
BASE_2050='--max-model-len 4096 --gpu-memory-utilization 0.80 --max-num-seqs 16 --enforce-eager --reasoning-parser qwen3 --enable-auto-tool-choice --tool-call-parser hermes --enable-prompt-tokens-details'
BASE=$( [ "$GPU" = 3050 ] && echo "$BASE_3050" || echo "$BASE_2050" )
WL='--trace traces/M.jsonl --mode replay --load closed --conc 8 --n 100'
mk() { printf 'EXP=%s\nSERVE_ARGS=%q\nWL_ARGS=%q\nENVV=%q\n' "$2" "$BASE $3" "${4:-$WL}" "${5:-}" > configs/$1.env; }
```

Step 1b.3 Sweep runner: sweep.sh

```
#!/usr/bin/env bash
# usage: ./sweep.sh cfg1 cfg2 ...   (3 repeats each)
for cfg in "$@"; do for rep in 1 2 3; do REP=$rep ./run_config.sh $cfg; done; done
```

Use 1 repeat while exploring (REP=1 ./run_config.sh name), 3 repeats for numbers that go in the write-up.

Step 1b.4 Quality probe: harness/quality_probe.py

Measures whether a setting (quantization, fp8 KV, speculation) changes outputs. Greedy decoding, so differences are real.

```
import json, sys, os
from openai import OpenAI
c = OpenAI(base_url='http://localhost:8000/v1', api_key=os.environ['KEY'])
ps = [json.loads(l) for l in open('traces/M.jsonl')][:60]
out = []
for s in ps:
    msgs = [{'role': 'user', 'content': (s['ctx'][:1500] + ' ' + s['task'])}]
    r = c.chat.completions.create(model='Qwen/Qwen3-0.6B', messages=msgs, temperature=0, max_tokens=64, logprobs=True, top_logprobs=1, extra_body={'chat_template_kwargs': {'enable_thinking': False}})
    out.append({'text': r.choices[0].message.content, 'lp': [t.logprob for t in r.choices[0].logprobs.content]})
json.dump(out, open(sys.argv[1], 'w'))
```

Compare script analysis/compare_probe.py:

```
import json, sys, numpy as np
a = json.load(open(sys.argv[1])); b = json.load(open(sys.argv[2]))
print('exact match', np.mean([x['text'] == y['text'] for x, y in zip(a, b)]))
print('first 40 chars match', np.mean([x['text'][:40] == y['text'][:40] for x, y in zip(a, b)]))
print('mean logprob base', np.mean([np.mean(x['lp']) for x in a]), 'other', np.mean([np.mean(y['lp']) for y in b]))
```

Manual inference-engineering experiments:

Completed until now

Phase 0 — Setup

- Created project directories
- Inspected GPU, memory, CPU, disk, driver, and CUDA environment
- Created/activated Python virtual environment
- Installed vLLM and supporting packages
- Downloaded/verified Qwen3-0.6B
- Started vLLM successfully on the RTX 3050
- Completed smoke test
- Verified reasoning output separation
- Verified non-thinking mode
- Verified tool calls
- Verified prompt-token and cache details
- Collected vLLM metric names

Phase 1 — Harness

- Created the text corpus
- Recorded corpus SHA-256
- Created harness/common.py
- Created harness/gen_trace.py
- Generated deterministic traces
- Verified identical trace hashes
- Generated workload classes A, B, C, and D
- Created and ran check_trace.py
- Measured prompt lengths and ideal cache hit rates
- Created harness/scrape.py
- Collected server-side metrics
- Created the mock server
- Tested closed-loop mock serving
- Tested open-loop mock serving
- Created harness/run.py
- Created analysis/summarize.py
- Created analysis/hitrate.py

Phase 1b — Automation

- Created run_config.sh
- Added repeat-specific output folders
- Added automatic vLLM startup and shutdown
- Added GPU monitoring
- Added metrics scraping
- Added metadata recording
- Added VRAM-after cleanup checks
- Created configs/mk.sh
- Created sweep.sh
- Completed an automated e0-base run
- Saved baseline logs and metrics

1. E0 baseline run :

```
REP=1 ./run_config.sh e0-base
done e0-base
(venv) alis@LAPTOP-N3QNJPKH:~/project/agentic-serving-lab$ ls -lh runs/3050/e0/e0-base/rep1
cat runs/3050/e0/e0-base/rep1/vram-after.txt
total 21M
-rw-r--r-- 1 alis alis    0 Nov 10 12:28 client.log
-rw-r--r-- 1 alis alis  312 Nov 10 12:28 config.env
-rw-r--r-- 1 alis alis 177K Nov 10 12:48 dmon.csv
-rw-r--r-- 1 alis alis  75K Nov 10 12:48 gpu.csv
-rw-r--r-- 1 alis alis  514 Nov 10 12:28 meta.json
-rw-r--r-- 1 alis alis  20M Nov 10 12:48 metrics.csv
-rw-r--r-- 1 alis alis 799K Nov 10 12:48 requests.jsonl
-rw-r--r-- 1 alis alis 3.5K Nov 10 12:29 server.log
-rw-r--r-- 1 alis alis    0 Nov 10 12:28 startup-facts.txt
-rw-r--r-- 1 alis alis    8 Nov 10 12:48 vram-after.txt
115 MiB
```

```
(venv) alis@LAPTOP-N3QNJPKH:~/project/agentic-serving-lab$ grep -c '"session_done"' runs/3050/e0/e0-base/rep1/requests.jsonl
grep -c '"error"' runs/3050/e0/e0-base/rep1/requests.jsonl
30
1
(venv) alis@LAPTOP-N3QNJPKH:~/project/agentic-serving-lab$ python analysis/summarize.py \
  runs/3050/e0/e0-base/rep1/requests.jsonl \
  | tee analysis/e0-base-rep1-summary.txt
 run  sessions  requests  errors  dur_s  ttft_p50  ttft_p99  itl_p50  itl_p95  e2e_p50  e2e_p99  sess_p50  sess_p99  out_tok_s  in_tok_s  json_ok  cached_frac  truncated
rep1        30       152       1 1068.5     0.986     3.716    0.183    0.216   44.973   109.01   225.687   722.511     34.814   321.577      1.0        0.808      0.105
```

2. E3 Experiment :

```
(venv) alis@LAPTOP-N3QNJPKH:~/project/agentic-serving-lab$ cd ~/project/agentic-serving-lab
source env/vars.sh
source configs/mk.sh
(venv) alis@LAPTOP-N3QNJPKH:~/project/agentic-serving-lab$ mk e3-on-P4 e3 '' \
  '--trace traces/M-P4.jsonl --mode replay --load closed --conc 2 --n 10 --warmup 1'
(venv) alis@LAPTOP-N3QNJPKH:~/project/agentic-serving-lab$ mk e3-off-P4 e3 '--no-enable-prefix-caching' \
  '--trace traces/M-P4.jsonl --mode replay --load closed --conc 2 --n 10 --warmup 1'
(venv) alis@LAPTOP-N3QNJPKH:~/project/agentic-serving-lab$ mk e3-off-P4 e3 '--no-enable-prefix-caching' \
  '--trace traces/M-P4.jsonl --mode replay --load closed --conc 2 --n 10 --warmup 1'
(venv) alis@LAPTOP-N3QNJPKH:~/project/agentic-serving-lab$ REP=1 ./run_config.sh e3-on-P4
done e3-on-P4
(venv) alis@LAPTOP-N3QNJPKH:~/project/agentic-serving-lab$ REP=1 ./run_config.sh e3-off-P4



done e3-off-P4
```

```
(venv) alis@LAPTOP-N3QNJPKH:~/project/agentic-serving-lab$ python analysis/summarize.py \
  runs/3050/e3/e3-on-P4/rep1/requests.jsonl \
  runs/3050/e3/e3-off-P4/rep1/requests.jsonl \
  | tee analysis/e3-P4-summary.txt
 run  sessions  requests  errors  dur_s  ttft_p50  ttft_p99  itl_p50  itl_p95  e2e_p50  e2e_p99  sess_p50  sess_p99  out_tok_s  in_tok_s  json_ok  cached_frac  truncated
rep1        14        57       0  742.3     0.303     1.837    0.049    0.076   10.927   35.365     3.853   220.926     15.045   137.718      1.0        0.769      0.018
rep1        10        45       0  657.1     1.638     5.056    0.086    0.106   25.575   54.694    62.326   316.072     17.334   138.675      1.0        0.000      0.133
```

```
(venv) alis@LAPTOP-N3QNJPKH:~/project/agentic-serving-lab$ python analysis/hitrate.py \
  runs/3050/e3/e3-on-P4/rep1/metrics.csv prefix_cache

python analysis/hitrate.py \
  runs/3050/e3/e3-off-P4/rep1/metrics.csv prefix_cache
vllm:external_prefix_cache_hits_created {engine="0",model_name="Qwen/Qwen3-0.6B"} delta 0.0
vllm:external_prefix_cache_hits_total {engine="0",model_name="Qwen/Qwen3-0.6B"} delta 0.0
vllm:external_prefix_cache_queries_created {engine="0",model_name="Qwen/Qwen3-0.6B"} delta 0.0
vllm:external_prefix_cache_queries_total {engine="0",model_name="Qwen/Qwen3-0.6B"} delta 0.0
vllm:prefix_cache_hits_created {engine="0",model_name="Qwen/Qwen3-0.6B"} delta 0.0
vllm:prefix_cache_hits_total {engine="0",model_name="Qwen/Qwen3-0.6B"} delta 73232.0
vllm:prefix_cache_queries_created {engine="0",model_name="Qwen/Qwen3-0.6B"} delta 0.0
vllm:prefix_cache_queries_total {engine="0",model_name="Qwen/Qwen3-0.6B"} delta 93254.0
vllm:external_prefix_cache_hits_created {engine="0",model_name="Qwen/Qwen3-0.6B"} delta 0.0
vllm:external_prefix_cache_hits_total {engine="0",model_name="Qwen/Qwen3-0.6B"} delta 0.0
vllm:external_prefix_cache_queries_created {engine="0",model_name="Qwen/Qwen3-0.6B"} delta 0.0
vllm:external_prefix_cache_queries_total {engine="0",model_name="Qwen/Qwen3-0.6B"} delta 0.0
vllm:prefix_cache_hits_created {engine="0",model_name="Qwen/Qwen3-0.6B"} delta 0.0
vllm:prefix_cache_hits_total {engine="0",model_name="Qwen/Qwen3-0.6B"} delta 0.0
vllm:prefix_cache_queries_created {engine="0",model_name="Qwen/Qwen3-0.6B"} delta 0.0
vllm:prefix_cache_queries_total {engine="0",model_name="Qwen/Qwen3-0.6B"} delta 0.0
(venv) alis@LAPTOP-N3QNJPKH:~/project/agentic-serving-lab$ python analysis/summarize.py \
  runs/3050/e3/e3-on-P4/rep1/requests.jsonl \
  runs/3050/e3/e3-off-P4/rep1/requests.jsonl \
```

```
  | tee analysis/e3-P4-summary.txt
 run  sessions  requests  errors  dur_s  ttft_p50  ttft_p99  itl_p50  itl_p95  e2e_p50  e2e_p99  sess_p50  sess_p99  out_tok_s  in_tok_s  json_ok  cached_frac  truncated
rep1        14        57       0  742.3     0.303     1.837    0.049    0.076   10.927   35.365     3.853   220.926     15.045   137.718      1.0        0.769      0.018
rep1        10        45       0  657.1     1.638     5.056    0.086    0.106   25.575   54.694    62.326   316.072     17.334   138.675      1.0        0.000      0.133
```

3. E4 experiment :

```
(venv) alis@LAPTOP-N3QNJPKH:~/project/agentic-serving-lab$ mk e4-bt512 e4 '--max-num-batched-tokens 512' \
  '--trace traces/M020.jsonl --mode replay --load closed --conc 2 --n 10 --warmup 1'

mk e4-bt4096 e4 '--max-num-batched-tokens 4096' \
  '--trace traces/M020.jsonl --mode replay --load closed --conc 2 --n 10 --warmup 1'
(venv) alis@LAPTOP-N3QNJPKH:~/project/agentic-serving-lab$ REP=1 ./run_config.sh e4-bt512
done e4-bt512
(venv) alis@LAPTOP-N3QNJPKH:~/project/agentic-serving-lab$ REP=1 ./run_config.sh e4-bt4096
done e4-bt4096
(venv) alis@LAPTOP-N3QNJPKH:~/project/agentic-serving-lab$ python analysis/summarize.py \
  runs/3050/e4/e4-bt512/rep1/requests.jsonl \
  runs/3050/e4/e4-bt4096/rep1/requests.jsonl \
  | tee analysis/e4-summary.txt
 run  sessions  requests  errors  dur_s  ttft_p50  ttft_p99  itl_p50  itl_p95  e2e_p50  e2e_p99  sess_p50  sess_p99  out_tok_s  in_tok_s  json_ok  cached_frac  truncated
rep1        10        45       0  630.1     0.644     3.039    0.080    0.106   20.916   55.029    68.669   277.316     17.161   144.624      1.0         0.79      0.133
rep1        10        45       0  599.8     0.664     2.430    0.085    0.098   20.954   49.123    65.184   315.675     18.120   151.916      1.0         0.79      0.089
```

4. E0 Concurrency :

```
(venv) alis@LAPTOP-N3QNJPKH:~/project/agentic-serving-lab$ cd ~/project/agentic-serving-lab
source env/vars.sh
source configs/mk.sh

for c in 1 2 4 8 16 32; do
  mk e0-c$c e0 '' \
    "--trace traces/M020.jsonl --mode replay --load closed --conc $c --n 20 --warmup 2"
done
(venv) alis@LAPTOP-N3QNJPKH:~/project/agentic-serving-lab$ ls configs/e0-c*.env
configs/e0-c1.env   configs/e0-c2.env   configs/e0-c4.env
configs/e0-c16.env  configs/e0-c32.env  configs/e0-c8.env
(venv) alis@LAPTOP-N3QNJPKH:~/project/agentic-serving-lab$ REP=1 ./run_config.sh e0-c1
REP=1 ./run_config.sh e0-c2
REP=1 ./run_config.sh e0-c4
REP=1 ./run_config.sh e0-c8
REP=1 ./run_config.sh e0-c16
REP=1 ./run_config.sh e0-c32
done e0-c1
done e0-c2
done e0-c4
done e0-c8
done e0-c16
done e0-c32
(venv) alis@LAPTOP-N3QNJPKH:~/project/agentic-serving-lab$
```

```
(venv) alis@LAPTOP-N3QNJPKH:~/project/agentic-serving-lab$ python analysis/summarize.py \
  runs/3050/e0/e0-c1/rep1/requests.jsonl \
  runs/3050/e0/e0-c2/rep1/requests.jsonl \
  runs/3050/e0/e0-c4/rep1/requests.jsonl \
  runs/3050/e0/e0-c8/rep1/requests.jsonl \
  runs/3050/e0/e0-c16/rep1/requests.jsonl \
  runs/3050/e0/e0-c32/rep1/requests.jsonl \
  | tee analysis/e0-concurrency-summary.txt
 run  sessions  requests  errors    dur_s  ttft_p50  ttft_p99  itl_p50  itl_p95  e2e_p50  e2e_p99  sess_p50  sess_p99  out_tok_s  in_tok_s  json_ok  cached_frac  truncated
rep1        27       118       0 10150.3     0.477     2.603    0.066    0.110   15.285   47.863     8.691   239.933      2.366    21.924      1.0        0.778      0.085
rep1        20        91       0  1271.8     0.711     2.931    0.113    0.153   27.016   63.960    61.076   425.847     15.246   141.802      1.0        0.779      0.066
rep1        20        91       0   729.4     0.720     3.017    0.130    0.167   28.830   78.226    75.766   477.231     26.881   247.254      1.0        0.779      0.121
rep1        20        91       0   627.1     1.037     3.378    0.170    0.216   40.039  106.551    90.423   582.455     33.545   287.588      1.0        0.779      0.143
rep1        20        91       0   296.6     0.593     4.034    0.070    0.161   19.647   72.274    75.910   287.480     67.668   608.019      1.0        0.779      0.099
rep1        20        91       0   214.7     0.519     2.258    0.061    0.082   16.276   40.308    36.468   205.183     91.845   840.067      1.0        0.779      0.055
(venv) alis@LAPTOP-N3QNJPKH:~/project/agentic-serving-lab$
```

5. E5 Experiment :

```
(venv) alis@LAPTOP-N3QNJPKH:~/project/agentic-serving-lab$ source env/vars.sh
source configs/mk.sh

mk e5-gmu080 e5 '--gpu-memory-utilization 0.80' \
  '--trace traces/M020.jsonl --mode replay --load closed --conc 2 --n 5 --warmup 1'

REP=1 ./run_config.sh e5-gmu080
done e5-gmu080
(venv) alis@LAPTOP-N3QNJPKH:~/project/agentic-serving-lab$ python analysis/summarize.py \
  runs/3050/e5/e5-gmu080/rep1/requests.jsonl \
  | tee analysis/e5-gmu080-summary.txt
 run  sessions  requests  errors  dur_s  ttft_p50  ttft_p99  itl_p50  itl_p95  e2e_p50  e2e_p99  sess_p50  sess_p99  out_tok_s  in_tok_s  json_ok  cached_frac  truncated
rep1         5        18       0  115.2     0.123     0.454    0.034    0.042    2.651    20.61     3.372   110.749     29.743   215.979      1.0        0.776      0.167
(venv) alis@LAPTOP-N3QNJPKH:~/project/agentic-serving-lab$
```

6. E6 Experiment :

```
(venv) alis@LAPTOP-N3QNJPKH:~/project/agentic-serving-lab$ source env/vars.sh
source configs/mk.sh

J='{"method":"ngram","num_speculative_tokens":3,"prompt_lookup_max":4,"prompt_lookup_min":2}'

mk e6-ng3 e6 "--speculative-config '$J'" \
  '--trace traces/M020.jsonl --mode replay --load closed --conc 2 --n 5 --warmup 1'

REP=1 ./run_config.sh e6-ng3
done e6-ng3
```

Summarize the run:
python analysis/summarize.py \
  runs/3050/e6/e6-ng3/rep1/requests.jsonl \
  | tee analysis/e6-ng3-summary.txt

```
(venv) alis@LAPTOP-N3QNJPKH:~/project/agentic-serving-lab$ python analysis/summarize.py \
  runs/3050/e6/e6-ng3/rep1/requests.jsonl \
  | tee analysis/e6-ng3-summary.txt
 run  sessions  requests  errors  dur_s  ttft_p50  ttft_p99  itl_p50  itl_p95  e2e_p50  e2e_p99  sess_p50  sess_p99  out_tok_s  in_tok_s  json_ok  cached_frac  truncated
rep1         5        18       0   87.1     0.113     0.413    0.029    0.044    2.677   16.588     2.454    83.776     32.596   285.458      1.0        0.776      0.111
```

Check whether speculative metrics were emitted:
python analysis/hitrate.py \
  runs/3050/e6/e6-ng3/rep1/metrics.csv spec

```
(venv) alis@LAPTOP-N3QNJPKH:~/project/agentic-serving-lab$ python analysis/hitrate.py \
  runs/3050/e6/e6-ng3/rep1/metrics.csv spec
vllm:spec_decode_num_accepted_tokens_created {engine="0",model_name="Qwen/Qwen3-0.6B"} delta 0.0
vllm:spec_decode_num_accepted_tokens_per_pos_created {engine="0",model_name="Qwen/Qwen3-0.6B",position="0"} delta 0.0
vllm:spec_decode_num_accepted_tokens_per_pos_created {engine="0",model_name="Qwen/Qwen3-0.6B",position="1"} delta 0.0
vllm:spec_decode_num_accepted_tokens_per_pos_created {engine="0",model_name="Qwen/Qwen3-0.6B",position="2"} delta 0.0
vllm:spec_decode_num_accepted_tokens_per_pos_total {engine="0",model_name="Qwen/Qwen3-0.6B",position="0"} delta 270.0
vllm:spec_decode_num_accepted_tokens_per_pos_total {engine="0",model_name="Qwen/Qwen3-0.6B",position="1"} delta 162.0
vllm:spec_decode_num_accepted_tokens_per_pos_total {engine="0",model_name="Qwen/Qwen3-0.6B",position="2"} delta 113.0
vllm:spec_decode_num_accepted_tokens_total {engine="0",model_name="Qwen/Qwen3-0.6B"} delta 545.0
vllm:spec_decode_num_draft_tokens_created {engine="0",model_name="Qwen/Qwen3-0.6B"} delta 0.0
vllm:spec_decode_num_draft_tokens_total {engine="0",model_name="Qwen/Qwen3-0.6B"} delta 1950.0
vllm:spec_decode_num_drafts_created {engine="0",model_name="Qwen/Qwen3-0.6B"} delta 0.0
vllm:spec_decode_num_drafts_total {engine="0",model_name="Qwen/Qwen3-0.6B"} delta 650.0
```

Compare against the baseline:
python analysis/summarize.py \
  runs/3050/e0/e0-base/rep1/requests.jsonl \
  runs/3050/e6/e6-ng3/rep1/requests.jsonl \
  | tee analysis/e6-vs-baseline.txt

```
(venv) alis@LAPTOP-N3QNJPKH:~/project/agentic-serving-lab$ python analysis/summarize.py \
  runs/3050/e0/e0-base/rep1/requests.jsonl \
  runs/3050/e6/e6-ng3/rep1/requests.jsonl \
  | tee analysis/e6-vs-baseline.txt
 run  sessions  requests  errors  dur_s  ttft_p50  ttft_p99  itl_p50  itl_p95  e2e_p50  e2e_p99  sess_p50  sess_p99  out_tok_s  in_tok_s  json_ok  cached_frac  truncated
rep1        30       152       1 1068.5     0.986     3.716    0.183    0.216   44.973  109.010   225.687   722.511     34.814   321.577      1.0        0.808      0.105
rep1         5        18       0   87.1     0.113     0.413    0.029    0.044    2.677   16.588     2.454    83.776     32.596   285.458      1.0        0.776      0.111
```

E6 conclusion :
N-gram speculative decoding was supported and achieved modest, variable acceptance. Its benefit must be judged by comparing throughput and latency against the baseline, while accounting for the V1 fallback and disabled async scheduling.

Key observations:

- N-gram speculation was enabled with 3 draft tokens.
- vLLM used the V1 model runner because V2 does not support n-gram speculation.
- Async scheduling was disabled.
- Mean accepted sequence length varied from about 1.6 to 2.4 tokens.
- Draft acceptance rate varied from about 19.5% to 45.5%.
- Per-position acceptance was highest for the first speculative token and declined for later tokens.
- Speculation produced accepted tokens, so it was not silently ignored.

```
_tokens_details': True, 'api_key': '***', 'max_model_len': 8192, 'enforce_eager': True, 'reasoning_parser': 'qwen3', 'gpu_memory_utilization': 0.75, 'max_num_seqs': 16, 'speculative_config':
 {'method': 'ngram', 'num_speculative_tokens': 3, 'prompt_lookup_max': 4, 'prompt_lookup_min': 2}}
(APIServer pid=129859) WARNING 11-10 18:06:30 [vllm.py:1596] Async scheduling not supported with ngram-based speculative decoding and will be disabled.
(APIServer pid=129859) WARNING 11-10 18:06:30 [vllm.py:2372] max_num_scheduled_tokens is set to 2048 based on the speculative decoding settings. This may lead to suboptimal performance. Cons
ider increasing max_num_batched_tokens to accommodate the additional draft token slots, or decrease num_speculative_tokens.
(APIServer pid=129859) WARNING 11-10 18:06:30 [vllm.py:754] Model Runner V2 does not yet support speculative method 'ngram'; using the V1 model runner instead.
(EngineCore pid=129994) INFO 11-10 18:06:41 [core.py:129] Initializing a V1 LLM engine (v0.31.0) with config: model='Qwen/Qwen3-0.6B', speculative_config=SpeculativeConfig(method='ngram', mo
del=None, num_spec_tokens=3), tokenizer='Qwen/Qwen3-0.6B', skip_tokenizer_init=False, tokenizer_mode=auto, revision=main, tokenizer_revision=main, trust_remote_code=False, dtype=torch.bfloat
16, max_seq_len=8192, download_dir=None, load_format=auto, tensor_parallel_size=1, pipeline_parallel_size=1, data_parallel_size=1, decode_context_parallel_size=1, dcp_comm_backend=ag_rs, dis
able_custom_all_reduce=False, quantization=None, quantization_config=None, enforce_eager=True, aux_output_config=AuxOutputConfig(enable_return_routed_experts=False, max_bytes=None), kv_cache
_dtype=auto, device_config=cuda, structured_outputs_config=StructuredOutputsConfig(backend='auto', disable_any_whitespace=False, disable_additional_properties=False, reasoning_parser='qwen3'
, reasoning_parser_plugin='', enable_in_reasoning=False), observability_config=ObservabilityConfig(show_hidden_metrics_for_version=None, otlp_traces_endpoint=None, collect_detailed_traces=No
ne, per_request_spec_decode_metrics='none', kv_cache_metrics=False, kv_cache_metrics_sample=0.01, cudagraph_metrics=False, enable_layerwise_nvtx_tracing=False, enable_mfu_metrics=False, enab
le_mm_processor_stats=False, enable_logging_iteration_details=False, jit_monitor_mode='warn', jit_monitor_verbose=False), seed=0, served_model_name=Qwen/Qwen3-0.6B, enable_prefix_caching=Tru
e, enable_chunked_prefill=True, pooler_config=None, compilation_config={'mode': <CompilationMode.NONE: 0>, 'debug_dump_path': None, 'cache_dir': '', 'compile_cache_save_format': 'binary', 'b
ackend': 'inductor', 'custom_ops': ['all'], 'ir_enable_torch_wrap': False, 'splitting_ops': [], 'compile_mm_encoder': False, 'cudagraph_mm_encoder': False, 'encoder_cudagraph_token_budgets':
 [], 'encoder_cudagraph_max_vision_items_per_batch': 0, 'encoder_cudagraph_max_frames_per_batch': None, 'compile_sizes': [], 'compile_ranges_endpoints': [2048], 'inductor_compile_config': {'
enable_auto_functionalized_v2': False, 'combo_kernels': True, 'benchmark_combo_kernel': True}, 'inductor_passes': {}, 'cudagraph_mode': <CUDAGraphMode.NONE: 0>, 'cudagraph_num_of_warmups': 0
, 'cudagraph_capture_sizes': [], 'cudagraph_copy_inputs': False, 'cudagraph_specialize_lora': True, 'use_inductor_graph_partition': False, 'pass_config': {'fuse_norm_quant': True, 'fuse_act_
quant': True, 'fuse_attn_quant': False, 'enable_sp': False, 'fuse_gemm_comms': False, 'fuse_allreduce_rms': False, 'enable_qk_norm_rope_fusion': False, 'fuse_rope_kvcache_cat_mla': False, 'f
use_act_padding': False, 'fuse_qk_norm_rope_kvcache': False}, 'max_cudagraph_capture_size': 0, 'dynamic_shapes_config': {'type': <DynamicShapesType.BACKED: 'backed'>, 'evaluate_guards': Fals
e, 'assume_32_bit_indexing': False}, 'local_cache_dir': None, 'fast_moe_cold_start': False, 'static_all_moe_layers': []}, kernel_config=KernelConfig(ir_op_priority=IrOpPriorityConfig(rms_nor
m=['vllm_c', 'native'], fused_add_rms_norm=['vllm_c', 'native'], gelu_and_mul_sparse=['triton', 'native']), enable_flashinfer_autotune=True, enable_cutedsl_warmup=True, enable_jit_warmup=Fal
se, moe_backend='auto', sparse_indexer_topk_backend='auto', linear_backend='auto', linear_backend_per_quant=None)
(EngineCore pid=129994) WARNING 11-10 18:06:42 [vllm.py:754] Model Runner V2 does not yet support speculative method 'ngram'; using the V1 model runner instead.
(EngineCore pid=129994) WARNING 11-10 18:06:49 [__init__.py:211] min_p and logit_bias parameters won't work with speculative decoding.
(EngineCore pid=129994) INFO 11-10 18:07:14 [gpu_model_runner.py:5286] Loading drafter model...
(EngineCore pid=129994) WARNING 11-10 18:07:32 [vllm.py:2372] max_num_scheduled_tokens is set to 2048 based on the speculative decoding settings. This may lead to suboptimal performance. Con
sider increasing max_num_batched_tokens to accommodate the additional draft token slots, or decrease num_speculative_tokens.
(APIServer pid=129859) INFO 11-10 18:08:12 [metrics.py:120] SpecDecoding metrics: Mean acceptance length: 2.20, Accepted throughput: 2.18 tokens/s, Drafted throughput: 5.46 tokens/s, Accepte
d: 85 tokens, Drafted: 213 tokens, Per-position acceptance rate: 0.535, 0.366, 0.296, Avg Draft acceptance rate: 39.9%
(APIServer pid=129859) INFO 11-10 18:08:22 [metrics.py:120] SpecDecoding metrics: Mean acceptance length: 1.70, Accepted throughput: 8.10 tokens/s, Drafted throughput: 34.50 tokens/s, Accept
ed: 81 tokens, Drafted: 345 tokens, Per-position acceptance rate: 0.383, 0.200, 0.122, Avg Draft acceptance rate: 23.5%
(APIServer pid=129859) INFO 11-10 18:08:32 [metrics.py:120] SpecDecoding metrics: Mean acceptance length: 1.59, Accepted throughput: 5.10 tokens/s, Drafted throughput: 26.10 tokens/s, Accept
ed: 51 tokens, Drafted: 261 tokens, Per-position acceptance rate: 0.368, 0.161, 0.057, Avg Draft acceptance rate: 19.5%
(APIServer pid=129859) INFO 11-10 18:08:42 [metrics.py:120] SpecDecoding metrics: Mean acceptance length: 1.66, Accepted throughput: 6.10 tokens/s, Drafted throughput: 27.90 tokens/s, Accept
ed: 61 tokens, Drafted: 279 tokens, Per-position acceptance rate: 0.366, 0.172, 0.118, Avg Draft acceptance rate: 21.9%
(APIServer pid=129859) INFO 11-10 18:08:52 [metrics.py:120] SpecDecoding metrics: Mean acceptance length: 1.86, Accepted throughput: 6.40 tokens/s, Drafted throughput: 22.20 tokens/s, Accept
ed: 64 tokens, Drafted: 222 tokens, Per-position acceptance rate: 0.446, 0.270, 0.149, Avg Draft acceptance rate: 28.8%
(APIServer pid=129859) INFO 11-10 18:09:02 [metrics.py:120] SpecDecoding metrics: Mean acceptance length: 1.62, Accepted throughput: 3.00 tokens/s, Drafted throughput: 14.40 tokens/s, Accept
ed: 30 tokens, Drafted: 144 tokens, Per-position acceptance rate: 0.292, 0.208, 0.125, Avg Draft acceptance rate: 20.8%
(APIServer pid=129859) INFO 11-10 18:09:12 [metrics.py:120] SpecDecoding metrics: Mean acceptance length: 2.37, Accepted throughput: 7.10 tokens/s, Drafted throughput: 15.60 tokens/s, Accept
ed: 71 tokens, Drafted: 156 tokens, Per-position acceptance rate: 0.519, 0.462, 0.385, Avg Draft acceptance rate: 45.5%
(APIServer pid=129859) INFO 11-10 18:09:22 [metrics.py:120] SpecDecoding metrics: Mean acceptance length: 1.89, Accepted throughput: 5.70 tokens/s, Drafted throughput: 19.20 tokens/s, Accept
ed: 57 tokens, Drafted: 192 tokens, Per-position acceptance rate: 0.438, 0.250, 0.203, Avg Draft acceptance rate: 29.7%
```

7. E7: Not completed
Reason: harness/run.py does not implement --think.

```
(venv) alis@LAPTOP-N3QNJPKH:~/project/agentic-serving-lab$ REP=1 ./run_config.sh e7-off
usage: run.py [-h] --trace TRACE [--mode {replay,live}] [--load {open,closed}]
              [--rate RATE] [--conc CONC] [--n N] [--warmup WARMUP]
              [--max-tokens MAX_TOKENS] [--base-url BASE_URL]
              [--api-key API_KEY] [--model MODEL] --out OUT
run.py: error: unrecognized arguments: --think off
done e7-off
(venv) alis@LAPTOP-N3QNJPKH:~/project/agentic-serving-lab$ source configs/mk.sh

mk e7-on e7 '' \
  '--trace traces/M020.jsonl --mode replay --load closed --conc 2 --n 5 --warmup 1 --think on'
(venv) alis@LAPTOP-N3QNJPKH:~/project/agentic-serving-lab$ REP=1 ./run_config.sh e7-on
usage: run.py [-h] --trace TRACE [--mode {replay,live}] [--load {open,closed}]
              [--rate RATE] [--conc CONC] [--n N] [--warmup WARMUP]
              [--max-tokens MAX_TOKENS] [--base-url BASE_URL]
              [--api-key API_KEY] [--model MODEL] --out OUT
run.py: error: unrecognized arguments: --think on
done e7-on
```

E7 did not run. The harness rejected:
unrecognized arguments: --think of

```
(venv) alis@LAPTOP-N3QNJPKH:~/project/agentic-serving-lab$ source env/vars.sh
source configs/mk.sh

mk e10-woff e10 '--cpu-offload-gb 1' \
  '--trace traces/M020.jsonl --mode replay --load closed --conc 2 --n 5 --warmup 1'

REP=1 ./run_config.sh e10-woff
done e10-woff
```

If no KV offload option appears, run the guide's fallback weight-offload test:
source env/vars.sh
source configs/mk.sh

mk e10-woff e10 '--cpu-offload-gb 1' \
  '--trace traces/M020.jsonl --mode replay --load closed --conc 2 --n 5 --warmup 1'

REP=1 ./run_config.sh e10-woff

After it completes:
python analysis/summarize.py \
  runs/3050/e10/e10-woff/rep1/requests.jsonl \
  | tee analysis/e10-woff-summary.txt
Then compare it with the baseline:
python analysis/summarize.py \
  runs/3050/e0/e0-base/rep1/requests.jsonl \
  runs/3050/e10/e10-woff/rep1/requests.jsonl \
  | tee analysis/e10-vs-baseline.txt

```
(venv) alis@LAPTOP-N3QNJPKH:~/project/agentic-serving-lab$ source env/vars.sh
source configs/mk.sh

mk e10-woff e10 '--cpu-offload-gb 1' \
  '--trace traces/M020.jsonl --mode replay --load closed --conc 2 --n 5 --warmup 1'

REP=1 ./run_config.sh e10-woff
done e10-woff
(venv) alis@LAPTOP-N3QNJPKH:~/project/agentic-serving-lab$ python analysis/summarize.py \
  runs/3050/e10/e10-woff/rep1/requests.jsonl \
  | tee analysis/e10-woff-summary.txt
 run  sessions  requests  errors  dur_s  ttft_p50  ttft_p99  itl_p50  itl_p95  e2e_p50  e2e_p99  sess_p50  sess_p99
out_tok_s  in_tok_s  json_ok  cached_frac  truncated
rep1         5        18       0  827.1     0.907     3.542    0.285    0.354   15.576  163.703    18.558   795.345
    3.614    30.071      1.0        0.776      0.167
(venv) alis@LAPTOP-N3QNJPKH:~/project/agentic-serving-lab$ python analysis/summarize.py \
  runs/3050/e0/e0-base/rep1/requests.jsonl \
  runs/3050/e10/e10-woff/rep1/requests.jsonl \
  | tee analysis/e10-vs-baseline.txt
 run  sessions  requests  errors  dur_s  ttft_p50  ttft_p99  itl_p50  itl_p95  e2e_p50  e2e_p99  sess_p50  sess_p99
out_tok_s  in_tok_s  json_ok  cached_frac  truncated
rep1        30       152       1 1068.5     0.986     3.716    0.183    0.216   44.973  109.010   225.687   722.511
   34.814   321.577      1.0        0.808      0.105
rep1         5        18       0  827.1     0.907     3.542    0.285    0.354   15.576  163.703    18.558   795.345
    3.614    30.071      1.0        0.776      0.167
(venv) alis@LAPTOP-N3QNJPKH:~/project/agentic-serving-lab$
```

E10 observation
Compared with the baseline:

- json_ok: remained perfect at 1.0.
- Errors: 0 in the offload run.
- TTFT p50: similar, 0.907 s versus 0.986 s.
- ITL p50: worsened from 0.183 s to 0.285 s.
- Output throughput: dropped from 34.8 to 3.6 tokens/s.
- Cached fraction: slightly lower, 77.6% versus 80.8%.
- Truncation increased from 10.5% to 16.7%.

Main conclusion:
CPU weight offload allowed the server to run, but PCIe/host-memory transfers caused a major decoding-throughput penalty. It is useful as a memory-saving fallback, not a speed optimization.
The comparison is directional because the baseline used 30 sessions while E10 used 5.

8. E11 Experiment :

Supported E11 options:
--scheduling-policy {fcfs,priority}
--async-scheduling
Run E11 priority scheduling

```
source env/vars.sh
source configs/mk.sh

mk e11-priority e11 '--scheduling-policy priority' \
  '--trace traces/M020.jsonl --mode replay --load closed --conc 2 --n 5 --warmup 1'

REP=1 ./run_config.sh e11-priority
```
Run the FCFS control:
```
mk e11-fcfs e11 '--scheduling-policy fcfs' \
  '--trace traces/M020.jsonl --mode replay --load closed --conc 2 --n 5 --warmup 1'

REP=1 ./run_config.sh e11-fcfs
```
Compare:
```
python analysis/summarize.py \
  runs/3050/e11/e11-fcfs/rep1/requests.jsonl \
  runs/3050/e11/e11-priority/rep1/requests.jsonl \
  | tee analysis/e11-summary.txt
```
Note: your current run.py does not support the guide's --prio workload argument, so both requests will use the default priority. This tests whether the scheduler accepts priority mode, but not differentiated request priorities.

E11 async scheduling comparison
```
mk e11-async e11 '--async-scheduling' \
  '--trace traces/M020.jsonl --mode replay --load closed --conc 2 --n 5 --warmup 1'

REP=1 ./run_config.sh e11-async
```
Compare it:
```
python analysis/summarize.py \
  runs/3050/e11/e11-fcfs/rep1/requests.jsonl \
  runs/3050/e11/e11-async/rep1/requests.jsonl \
  | tee analysis/e11-async-summary.txt
```

```
(venv) alis@LAPTOP-N3QNJPKH:~/project/agentic-serving-lab$ mk e11-fcfs e11 '--scheduling-policy fcfs' \
  '--trace traces/M020.jsonl --mode replay --load closed --conc 2 --n 5 --warmup 1'

REP=1 ./run_config.sh e11-fcfs
done e11-fcfs
(venv) alis@LAPTOP-N3QNJPKH:~/project/agentic-serving-lab$ python analysis/summarize.py \
  runs/3050/e11/e11-fcfs/rep1/requests.jsonl \
  runs/3050/e11/e11-priority/rep1/requests.jsonl \
  | tee analysis/e11-summary.txt
 run  sessions  requests  errors  dur_s  ttft_p50  ttft_p99  itl_p50  itl_p95  e2e_p50  e2e_p99  sess_p50  sess_
p99  out_tok_s  in_tok_s  json_ok  cached_frac  truncated
rep1         5         18       0  166.0     0.388     0.733    0.051    0.075    3.217   37.495     3.857   159.
591     16.746   149.813      1.0        0.776      0.111
rep1         5         18       0  180.2     0.306     0.634    0.049    0.069    2.497   29.397     2.743   173.
218     17.411   137.992      1.0        0.776      0.056
(venv) alis@LAPTOP-N3QNJPKH:~/project/agentic-serving-lab$ mk e11-async e11 '--async-scheduling' \
  '--trace traces/M020.jsonl --mode replay --load closed --conc 2 --n 5 --warmup 1'

REP=1 ./run_config.sh e11-async
done e11-async
(venv) alis@LAPTOP-N3QNJPKH:~/project/agentic-serving-lab$ python analysis/summarize.py \
  runs/3050/e11/e11-fcfs/rep1/requests.jsonl \
  runs/3050/e11/e11-async/rep1/requests.jsonl \
  | tee analysis/e11-async-summary.txt
 run  sessions  requests  errors  dur_s  ttft_p50  ttft_p99  itl_p50  itl_p95  e2e_p50  e2e_p99  sess_p50  sess_
p99  out_tok_s  in_tok_s  json_ok  cached_frac  truncated
rep1         5         18       0  166.0     0.388     0.733    0.051    0.075    3.217   37.495     3.857   159.
591     16.746   149.813      1.0        0.776      0.111
rep1         5         18       0  148.4     0.234     0.573    0.037    0.057    2.650   29.633     3.228   142.
694     22.172   167.559      1.0        0.776      0.167
(venv) alis@LAPTOP-N3QNJPKH:~/project/agentic-serving-lab$
```

E11 completed successfully.

Priority vs FCFS

Priority scheduling showed:

- Lower TTFT p50: 0.306s vs 0.388s
- Lower TTFT p99: 0.634s vs 0.733s
- Lower ITL p95: 0.069s vs 0.075s
- Higher output throughput: 17.4 vs 16.7 tokens/s
- Zero errors in both runs
- Same cache fraction: 77.6%

Because your workload did not assign different request priorities, this is only a scheduler-mode comparison—not a true interactive-priority test.

Async scheduling

Async scheduling improved most latency metrics:

- TTFT p50: 0.234s
- TTFT p99: 0.573s
- ITL p95: 0.057s
- Output throughput: 22.2 tokens/s
- Zero errors

However, truncation increased to 16.7%, so the speedup came with a quality/completion trade-off in this small run.

Record:

Async scheduling improved responsiveness and throughput on the RTX 3050, but the short sample showed more output truncation. Priority scheduling also improved latency slightly, though differentiated priorities were not tested.

9. E13: stress and failure testing.

1. Overload test

```
source env/vars.sh
source configs/mk.sh

mk e13-over e13 " \
  '--trace traces/M020.jsonl --mode replay --load open --rate 0.2 --n 10 --warmup 1'

REP=1 ./run_config.sh e13-over
Inspect errors and queue behavior:
grep -R '"error"' runs/3050/e13/e13-over/rep1
grep -iE 'Running:|Waiting:|preempt' \
  runs/3050/e13/e13-over/rep1/server.log
```

2. Long-prompt test

```
python harness/gen_trace.py \
  --n 20 \
  --mix D:1 \
  --scale 2 \
  --out traces/Dbig.jsonl
mk e13-toolong e13 " \
  '--trace traces/Dbig.jsonl --mode replay --load closed --conc 2 --n 5 --warmup 1'

REP=1 ./run_config.sh e13-toolong
Inspect:
grep -R '"error"' runs/3050/e13/e13-toolong/rep1
```

3. Summarize E13

```
python analysis/summarize.py \
  runs/3050/e13/e13-over/rep1/requests.jsonl \
  runs/3050/e13/e13-toolong/rep1/requests.jsonl \
  | tee analysis/e13-summary.txt
```

```
(venv) alis@LAPTOP-N3QNJPKH:~/project/agentic-serving-lab$ python analysis/summarize.py \
  runs/3050/e13/e13-over/rep1/requests.jsonl \
  runs/3050/e13/e13-toolong/rep1/requests.jsonl \
  | tee analysis/e13-summary.txt
 run  sessions  requests  errors  dur_s  ttft_p50  ttft_p99  itl_p50  itl_p95  e2e_p50  e2e_p99  sess_p50  sess_p99  out_tok_s  in_tok_s  json_ok  cached_frac  truncated
rep1        10        45       0  412.2     0.660     2.578    0.078    0.149   26.622   64.281    67.067   347.001     25.213   221.066      1.0         0.79      0.044
rep1         5         1       4    4.0     3.253     3.253    0.030    0.030    4.014    4.014     0.086     3.863      6.474  1559.204      1.0         0.00      0.000
(venv) alis@LAPTOP-N3QNJPKH:~/project/agentic-serving-lab$
```

Key finding

- The overload test remained stable with no recorded errors.
- The long-prompt test produced 4 context-length failures because the 8192-token limit did not leave enough room for the requested 512 output tokens.
- The long-prompt failures were correctly captured by the harness rather than crashing the run.

E13 completed and produced useful findings.

Overload test

sessions: 10
errors: 0
output throughput: 25.2 tokens/s
cached fraction: 79.0%
truncated: 4.4%
The server stayed stable under this short overload test. It did not produce client-visible errors.
Long-prompt test
sessions: 5
errors: 4
The failures were expected context-limit errors:
prompt >= 7681 tokens
requested output = 512 tokens
context limit = 8192 tokens
This confirms that your harness records oversized requests correctly instead of crashing.

10. EP14 Experiment :

E14: final configuration. Use the working baseline plus async scheduling:
source env/vars.sh
source configs/mk.sh

mk e14-final e14 '--async-scheduling' \
  '--trace traces/M020.jsonl --mode replay --load closed --conc 2 --n 10 --warmup 1'

REP=1 ./run_config.sh e14-final
After it completes:
python analysis/summarize.py \
  runs/3050/e14/e14-final/rep1/requests.jsonl \
  | tee analysis/e14-final-summary.txt

```
(venv) alis@LAPTOP-N3QNJPKH:~/project/agentic-serving-lab$ source env/vars.sh
source configs/mk.sh

mk e14-final e14 '--async-scheduling' \
  '--trace traces/M020.jsonl --mode replay --load closed --conc 2 --n 10 --warmup 1'

REP=1 ./run_config.sh e14-final
done e14-final
(venv) alis@LAPTOP-N3QNJPKH:~/project/agentic-serving-lab$ python analysis/summarize.py \
  runs/3050/e14/e14-final/rep1/requests.jsonl \
  | tee analysis/e14-final-summary.txt
 run  sessions  requests  errors  dur_s  ttft_p50  ttft_p99  itl_p50  itl_p95  e2e_p50  e2e_p99  sess_p50  sess_p99  out_tok_s  in_tok_s  json_ok  cached_frac  truncated
rep1        10        45       0  581.5     0.593     2.305     0.08    0.103   19.567   53.621    35.283   276.584     18.318   156.704      1.0         0.79      0.156
(venv) alis@LAPTOP-N3QNJPKH:~/project/agentic-serving-lab$
```

Metric	E14 final
Sessions	10
Requests	45
Errors	0
Duration	581.5 s
TTFT p50	0.593 s
TTFT p99	2.305 s
ITL p50	0.080 s

Metric	E14 final
ITL p95	0.103 s
E2E p50	19.567 s
E2E p99	53.621 s
Session p50	35.283 s
Session p99	276.584 s
Output throughput	18.318 tokens/s
Input throughput	156.704 tokens/s
Tool JSON validity	100%
Cached fraction	79.0%
Truncated	15.6%

Main result: the final async-scheduling configuration completed without errors and preserved perfect tool-call JSON validity. However, 15.6% of requests were truncated, so the output-token/context-limit issue remains.
Do not compare this directly with the E0 baseline as a definitive speedup because the workloads differ: E0 used 30 sessions at concurrency 8, while E14 used 10 sessions at concurrency 2. This is an exploratory final-candidate result.

111. E0 concurrency

```
    "--trace traces/M020.jsonl --mode replay --load closed --conc $c --n 5 --warmup 1"
  REP=1 ./run_config.sh e0-c$c
done
done e0-c1
done e0-c2
done e0-c4
done e0-c8
done e0-c16
(venv) alis@LAPTOP-N3QNJPKH:~/project/agentic-serving-lab$ python analysis/summarize.py \
  runs/3050/e0/e0-c*/rep1/requests.jsonl \
  | tee analysis/e0-concurrency-speedrun.txt
 run  sessions  requests  errors   dur_s  ttft_p50  ttft_p99  itl_p50  itl_p95  e2e_p50  e2e_p99  sess_p50  sess_p99  out_tok_s  in_tok_s  json_ok  cached_frac
  truncated
rep1        32       136       0 10150.3     0.379     2.509    0.063    0.110   12.395   47.276     6.585   237.175      2.607    24.374      1.0        0.778
     0.074
rep1        25       109       0   296.6     0.463     4.008    0.063    0.159   14.601   71.547     8.300   285.082     76.761   691.871      1.0        0.779
     0.092
rep1        25       109       0  1271.8     0.631     2.858    0.107    0.152   23.593   62.928    16.273   417.262     17.620   161.358      1.0        0.779
     0.064
rep1        20        91       0   214.7     0.519     2.258    0.061    0.082   16.276   40.308    36.468   205.183     91.845   840.067      1.0        0.779
     0.055
rep1        25       109       0   729.4     0.650     2.911    0.116    0.167   24.247   77.453    14.394   465.619     29.906   281.352      1.0        0.779
     0.101
rep1        25       109       0   627.1     0.890     3.299    0.152    0.214   32.067  106.275    20.070   572.569     37.428   327.249      1.0        0.779
     0.119
(venv) alis@LAPTOP-N3QNJPKH:~/project/agentic-serving-lab$
```

12. E3 P16 prefix caching

```
mk e3-on-P16 e3 " \
  '--trace traces/M-P16.jsonl --mode replay --load closed --conc 2 --n 5 --warmup 1'

mk e3-off-P16 e3 '--no-enable-prefix-caching' \
  '--trace traces/M-P16.jsonl --mode replay --load closed --conc 2 --n 5 --warmup 1'

REP=1 ./run_config.sh e3-on-P16
REP=1 ./run_config.sh e3-off-P16
python analysis/hitrate.py \
  runs/3050/e3/e3-on-P16/rep1/metrics.csv prefix_cache

python analysis/hitrate.py \
  runs/3050/e3/e3-off-P16/rep1/metrics.csv prefix_cache
```

```
(venv) alis@LAPTOP-N3QNJPKH:~/project/agentic-serving-lab$ mk e3-on-P16 e3 '' \
  '--trace traces/M-P16.jsonl --mode replay --load closed --conc 2 --n 5 --warmup 1'

mk e3-off-P16 e3 '--no-enable-prefix-caching' \
  '--trace traces/M-P16.jsonl --mode replay --load closed --conc 2 --n 5 --warmup 1'

REP=1 ./run_config.sh e3-on-P16
REP=1 ./run_config.sh e3-off-P16
done e3-on-P16
done e3-off-P16
(venv) alis@LAPTOP-N3QNJPKH:~/project/agentic-serving-lab$ python analysis/hitrate.py \
  runs/3050/e3/e3-on-P16/rep1/metrics.csv prefix_cache

python analysis/hitrate.py \
  runs/3050/e3/e3-off-P16/rep1/metrics.csv prefix_cache
vllm:external_prefix_cache_hits_created {engine="0",model_name="Qwen/Qwen3-0.6B"} delta 0.0
vllm:external_prefix_cache_hits_total {engine="0",model_name="Qwen/Qwen3-0.6B"} delta 0.0
vllm:external_prefix_cache_queries_created {engine="0",model_name="Qwen/Qwen3-0.6B"} delta 0.0
vllm:external_prefix_cache_queries_total {engine="0",model_name="Qwen/Qwen3-0.6B"} delta 0.0
vllm:prefix_cache_hits_created {engine="0",model_name="Qwen/Qwen3-0.6B"} delta 0.0
vllm:prefix_cache_hits_total {engine="0",model_name="Qwen/Qwen3-0.6B"} delta 60992.0
vllm:prefix_cache_queries_created {engine="0",model_name="Qwen/Qwen3-0.6B"} delta 0.0
vllm:prefix_cache_queries_total {engine="0",model_name="Qwen/Qwen3-0.6B"} delta 79244.0
vllm:external_prefix_cache_hits_created {engine="0",model_name="Qwen/Qwen3-0.6B"} delta 0.0
vllm:external_prefix_cache_hits_total {engine="0",model_name="Qwen/Qwen3-0.6B"} delta 0.0
vllm:external_prefix_cache_queries_created {engine="0",model_name="Qwen/Qwen3-0.6B"} delta 0.0
vllm:external_prefix_cache_queries_total {engine="0",model_name="Qwen/Qwen3-0.6B"} delta 0.0
vllm:prefix_cache_hits_created {engine="0",model_name="Qwen/Qwen3-0.6B"} delta 0.0
vllm:prefix_cache_hits_total {engine="0",model_name="Qwen/Qwen3-0.6B"} delta 0.0
vllm:prefix_cache_queries_created {engine="0",model_name="Qwen/Qwen3-0.6B"} delta 0.0
vllm:prefix_cache_queries_total {engine="0",model_name="Qwen/Qwen3-0.6B"} delta 0.0
(venv) alis@LAPTOP-N3QNJPKH:~/project/agentic-serving-lab$
```

13. E4 batching sweep

```
for t in 512 2048 4096 8192; do
  mk e4-bt$t e4 "--max-num-batched-tokens $t" \
    "--trace traces/M020.jsonl --mode replay --load closed --conc 2 --n 5 --warmup 1"
  REP=1 ./run_config.sh e4-bt$t
done
python analysis/summarize.py \
  runs/3050/e4/e4-bt*/rep1/requests.jsonl \
  | tee analysis/e4-speedrun.txt
```

```
(venv) alis@LAPTOP-N3QNJPKH:~/project/agentic-serving-lab$ for t in 512 2048 4096 8192; do
  mk e4-bt$t e4 "--max-num-batched-tokens $t" \
    "--trace traces/M020.jsonl --mode replay --load closed --conc 2 --n 5 --warmup 1"
  REP=1 ./run_config.sh e4-bt$t
done
done e4-bt512
done e4-bt2048
done e4-bt4096
done e4-bt8192
(venv) alis@LAPTOP-N3QNJPKH:~/project/agentic-serving-lab$ python analysis/summarize.py \
  runs/3050/e4/e4-bt*/rep1/requests.jsonl \
  | tee analysis/e4-speedrun.txt
 run  sessions  requests  errors  dur_s  ttft_p50  ttft_p99  itl_p50  itl_p95  e2e_p50  e2e_p99  sess_p50  sess_p99  out_tok_s  in_tok_s  json_ok  cached_frac  truncated
rep1         5        18       0  129.9     0.137     0.629    0.035    0.049    2.025   25.696     2.309   124.896     25.000   191.496      1.0        0.776      0.167
rep1        15        63       0  599.8     0.501     2.226    0.081    0.096   16.620   48.821     6.364   311.971     22.446   193.380      1.0        0.787      0.079
rep1        15        63       0  630.1     0.474     2.716    0.075    0.105   16.848   54.834     7.142   274.214     21.913   184.097      1.0        0.787      0.143
rep1         5        18       0  208.3     0.278     0.807    0.052    0.097    3.751   47.335     4.265   200.229     14.297   119.401      1.0        0.776      0.000
```

14. E8 compiled mode

Edit the config and remove --enforce-eager:
nano configs/e8-compiled.env
Then run:
REP=2 ./run_config.sh e8-compiled
Check:
grep -iE 'compile|cudagraph|eager|graph' \
  runs/3050/e8/e8-compiled/rep2/server.log

```
(venv) alis@LAPTOP-N3QNJPKH:~/project/agentic-serving-lab$ python analysis/summarize.py \
  runs/3050/e8/e8-compiled/rep2/requests.jsonl \
  | tee analysis/e8-compiled-summary.txt
 run  sessions  requests  errors  dur_s  ttft_p50  ttft_p99  itl_p50  itl_p95  e2e_p50  e2e_p99  sess_p50  sess_p99  out_tok_s  in_tok_s  json_ok  cached_frac
 truncated
rep2         5        18       0   75.1     0.092     0.317    0.018    0.033     1.38   14.762     1.567    72.234     42.949   331.011      1.0        0.776
     0.222
```

E8 completed successfully and this time it genuinely tested compiled mode.
Key observations:

- enforce_eager=False
- vLLM compilation mode enabled
- Torch compilation took about 35 seconds
- CUDA graphs were captured successfully
- CUDA graph memory used about 0.04 GiB
- Effective KV-cache capacity was slightly reduced because compiled graphs reserve memory
- No startup failure occurred

This confirms your RTX 3050 supports vLLM compilation and CUDA graphs with the current configuration.

Compare with the eager-mode control:
python analysis/summarize.py \
  runs/3050/e8/e8-compiled/rep1/requests.jsonl \
  runs/3050/e8/e8-compiled/rep2/requests.jsonl \
  | tee analysis/e8-eager-vs-compiled.txt

```
(venv) alis@LAPTOP-N3QNJPKH:~/project/agentic-serving-lab$ python analysis/summarize.py \
  runs/3050/e8/e8-compiled/rep1/requests.jsonl \
  runs/3050/e8/e8-compiled/rep2/requests.jsonl \
  | tee analysis/e8-eager-vs-compiled.txt
 run  sessions  requests  errors  dur_s  ttft_p50  ttft_p99  itl_p50  itl_p95  e2e_p50  e2e_p99  sess_p50  sess_p99  out_tok_s  in_tok_s  json_ok  cached_frac
 truncated
rep1         5        18       0  159.6     0.364     0.928    0.061    0.077    3.674   33.963     4.067   153.514     16.897   155.815      1.0        0.776
     0.056
rep2         5        18       0   75.1     0.092     0.317    0.018    0.033    1.380   14.762     1.567    72.234     42.949   331.011      1.0        0.776
     0.222
(venv) alis@LAPTOP-N3QNJPKH:~/project/agentic-serving-lab$
```

E14 ablation waterfall. We'll test whether async scheduling actually earns its place.
Create three short configurations:
source env/vars.sh
source configs/mk.sh

mk e14-base e14 " \
  '--trace traces/M020.jsonl --mode replay --load closed --conc 2 --n 5 --warmup 1'

mk e14-add-async e14 '--async-scheduling' \
  '--trace traces/M020.jsonl --mode replay --load closed --conc 2 --n 5 --warmup 1'
Run the baseline:
REP=1 ./run_config.sh e14-base
Run the optimized version:
REP=1 ./run_config.sh e14-add-async
Compare:
python analysis/summarize.py \
  runs/3050/e14/e14-base/rep1/requests.jsonl \
  runs/3050/e14/e14-add-async/rep1/requests.jsonl \
  | tee analysis/e14-ablation-summary.txt

```
(venv) alis@LAPTOP-N3QNJPKH:~/project/agentic-serving-lab$ REP=1 ./run_config.sh e14-add-async
done e14-add-async
(venv) alis@LAPTOP-N3QNJPKH:~/project/agentic-serving-lab$ python analysis/summarize.py \
  runs/3050/e14/e14-base/rep1/requests.jsonl \
  runs/3050/e14/e14-add-async/rep1/requests.jsonl \
  | tee analysis/e14-ablation-summary.txt
 run  sessions  requests  errors  dur_s  ttft_p50  ttft_p99  itl_p50  itl_p95  e2e_p50  e2e_p99  sess_p50  sess_p99  out_tok_s  in_tok_s  json_ok  cached_frac  truncated
rep1         5        18       0   96.4     0.152     0.375    0.032    0.045    2.179   21.280     2.538    92.685     29.700   258.092      1.0        0.776      0.056
rep1         5        18       0  120.3     0.267     0.566    0.035    0.059    2.606   20.412     3.339   115.678     21.521   206.737      1.0        0.776      0.000
(venv) alis@LAPTOP-N3QNJPKH:~/project/agentic-serving-lab$
```

Final Summary Report :

```
(venv) alis@LAPTOP-N3QNJPKH:~/project/agentic-serving-lab$ python analysis/summarize.py   runs/3050/e0/e0-base/rep1/requests.jsonl   runs/3050/e0/e0-c1/rep1/requests.jsonl   runs/3050/e0/e0-c2/rep1/requests.jsonl   runs/3050/e0/e0-c4/rep1/requests.jsonl   runs/3050/e0/e0-c8/rep1/requests.jsonl   runs/3050/e0/e0-c16/rep1/requests.jsonl   runs/3050/e0/e0-c32/rep1/requests.jsonl   runs/3050/e3/*/rep1/requests.jsonl   runs/3050/e4/*/rep1/requests.jsonl   runs/3050/e5/e5-gmu080/rep1/requests.jsonl   runs/3050/e6/e6-ng3/rep1/requests.jsonl   runs/3050/e8/e8-compiled/rep1/requests.jsonl   runs/3050/e8/e8-compiled/rep2/requests.jsonl   runs/3050/e9/e9-nccl/rep1/requests.jsonl   runs/3050/e10/e10-woff/rep1/requests.jsonl   runs/3050/e11/*/rep1/requests.jsonl   runs/3050/e13/e13-over/rep1/requests.jsonl   runs/3050/e13/e13-toolong/rep1/requests.jsonl   runs/3050/e14/*/rep1/requests.jsonl   | tee analysis/all-results-summary.txt
 run  sessions  requests  errors   dur_s  ttft_p50  ttft_p99  itl_p50  itl_p95  e2e_p50  e2e_p99  sess_p50  sess_p99  out_tok_s  in_tok_s  json_ok  cached_frac  truncated
rep1        30       152       1  1068.5     0.986     3.716    0.183    0.216   44.973  109.010   225.687   722.511     34.814   321.577      1.0        0.808      0.105
rep1        32       136       0 10150.3     0.379     2.509    0.063    0.110   12.395   47.276     6.585   237.175      2.607    24.374      1.0        0.778      0.074
rep1        25       109       0  1271.8     0.631     2.858    0.107    0.152   23.593   62.928    16.273   417.262     17.620   161.358      1.0        0.779      0.064
rep1        25       109       0   729.4     0.650     2.911    0.116    0.167   24.247   77.453    14.394   465.619     29.906   281.352      1.0        0.779      0.101
rep1        25       109       0   627.1     0.890     3.299    0.152    0.214   32.067  106.275    20.070   572.569     37.428   327.249      1.0        0.779      0.119
rep1        25       109       0   296.6     0.463     4.008    0.063    0.159   14.601   71.547     8.300   285.082     76.761   691.871      1.0        0.779      0.092
rep1        20        91       0   214.7     0.519     2.258    0.061    0.082   16.276   40.308    36.468   205.183     91.845   840.067      1.0        0.779      0.055
rep1         5        31       1   283.0     0.648     1.693    0.046    0.127   13.910   40.052   105.837   138.097     34.338   247.684      1.0        0.000      0.290
rep1        10        45       0   657.1     1.638     5.056    0.086    0.106   25.575   54.694    62.326   316.072     17.334   138.675      1.0        0.000      0.133
rep1         5        31       1   415.9     0.310     1.309    0.069    0.121   21.349   71.073   170.603   237.907     22.845   168.566      1.0        0.787      0.161
rep1        14        57       0   742.3     0.303     1.837    0.049    0.076   10.927   35.365     3.853   220.926     15.045   137.718      1.0        0.769      0.018
rep1         5        18       0   129.9     0.137     0.629    0.035    0.049    2.025   25.696     2.309   124.896     25.000   191.496      1.0        0.776      0.167
rep1        15        63       0   599.8     0.501     2.226    0.081    0.096   16.620   48.821     6.364   311.971     22.446   193.380      1.0        0.787      0.079
rep1        15        63       0   630.1     0.474     2.716    0.075    0.105   16.848   54.834     7.142   274.214     21.913   184.097      1.0        0.787      0.143
rep1         5        18       0   208.3     0.278     0.807    0.052    0.097    3.751   47.335     4.265   200.229     14.297   119.401      1.0        0.776      0.000
rep1         5        18       0   115.2     0.123     0.454    0.034    0.042    2.651   20.610     3.372   110.749     29.743   215.979      1.0        0.776      0.167
rep1         5        18       0    87.1     0.113     0.413    0.029    0.044    2.677   16.588     2.454    83.776     32.596   285.458      1.0        0.776      0.111
rep1         5        18       0   159.6     0.364     0.928    0.061    0.077    3.674   33.963     4.067   153.514     16.897   155.815      1.0        0.776      0.056
rep2         5        18       0    75.1     0.092     0.317    0.018    0.033    1.380   14.762     1.567    72.234     42.949   331.011      1.0        0.776      0.222
rep1        30       152       1   869.3     0.933     3.166    0.180    0.221   36.508  113.590   213.794   620.175     42.293   395.237      1.0        0.805      0.112
rep1         5        18       0   827.1     0.907     3.542    0.285    0.354   15.576  163.703    18.558   795.345      3.614    30.071      1.0        0.776      0.167
rep1         5        18       0   148.4     0.234     0.573    0.037    0.057    2.650   29.633     3.228   142.694     22.172   167.559      1.0        0.776      0.167
rep1         5        18       0   166.0     0.388     0.733    0.051    0.075    3.217   37.495     3.857   159.591     16.746   149.813      1.0        0.776      0.111
rep1         5        18       0   180.2     0.306     0.634    0.049    0.069    2.497   29.397     2.743   173.218     17.411   137.992      1.0        0.776      0.056
rep1        10        45       0   412.2     0.660     2.578    0.078    0.149   26.622   64.281    67.067   347.001     25.213   221.066      1.0        0.790      0.044
rep1         5         1       4     4.0     3.253     3.253    0.030    0.030    4.014    4.014     0.086     3.863      6.474  1559.204      1.0        0.000      0.000
rep1         5        18       0   120.3     0.267     0.566    0.035    0.059    2.606   20.412     3.339   115.678     21.521   206.737      1.0        0.776      0.000
rep1         5        18       0    96.4     0.152     0.375    0.032    0.045    2.179   21.280     2.538    92.685     29.700   258.092      1.0        0.776      0.056
rep1        10        45       0   581.5     0.593     2.305    0.080    0.103   19.567   53.621    35.283   276.584     18.318   156.704      1.0        0.790      0.156
(venv) alis@LAPTOP-N3QNJPKH:~/project/agentic-serving-lab$
```

Important findings:

- E0 concurrency scaling: throughput increased strongly with concurrency:
  - c1: ~2.6 tok/s
  - c2: ~17.6 tok/s
  - c4: ~29.9 tok/s
  - c8: ~37.4 tok/s
  - c16: ~76.8 tok/s
  - c32: ~91.8 tok/s

Higher concurrency improved throughput, but session latency and duration varied greatly.

- E5 weight offload: completed but was much slower:
  - Output throughput: ~3.6 tok/s
  - ITL p50: ~0.285 s
  - Confirms CPU offload is a memory-saving fallback, not a performance optimization.
- E6 speculative decoding: completed successfully:
  - Output throughput: ~32.6 tok/s
  - No errors
  - Speculative acceptance was confirmed in the server logs.
- E8 compiled mode: successful:
  - Eager control: ~16.9 tok/s
  - Compiled mode: ~42.9 tok/s
  - Compiled mode substantially improved this short run.
- E10 weight offload: slower than baseline, as expected.
- E11 scheduling:
  - Async scheduling: ~22.2 tok/s

    - FCFS: ~16.7 tok/s
    - Priority: ~17.4 tok/s
    - Async scheduling performed best in the small comparison.
- E13 overload: completed with zero errors.
- E13 long prompt: correctly produced 4 context-length errors.
- E13 forced OOM: correctly failed during startup with a clear VRAM error.
- E14 final candidate: completed with zero errors, 100% tool JSON validity, and ~79% cached prompt fraction.

One limitation: every row is labeled rep1 because summarize.py currently uses only the parent directory name. The rows are still in the same order as the file paths supplied to the command, but the summary would be clearer if the run-label logic were improved.

Experiment Results

Experiment	Configuration / workload	Status	Measured output	Interpretation
Phase 0	GPU, CUDA, CPU, memory and disk inspection	Complete	RTX 3050 6 GB; driver 610.88; CUDA 13.3	Established the hardware ceiling and available VRAM.
Phase 1	Corpus generation	Complete	4,674,931 characters in traces/corpus.txt	Provided repeatable tool-result text.
Phase 1	Trace determinism	Complete	M.jsonl and M2.jsonl SHA-256 hashes matched	The workload generator is reproducible.
Phase 1	Trace check: M.jsonl	Complete	1,560 requests; max prompt 11,657; 61 over 8,192; ideal hit rate 0.875	Original workload was too large for the selected context.
Phase 1	Trace check: M020.jsonl	Complete	1,560 requests; max prompt 9,668; 12 over limit; ideal hit rate 0.818	Scale 0.20 reduced, but did not eliminate, context pressure.
Phase 1	Project diversity: 8 projects	Complete	Ideal hit rate 0.813	More project contexts slightly reduced prefix reuse.
Phase 1	Project diversity: 16 projects	Complete	Ideal hit rate 0.812	More unique prefixes produced slightly less cache reuse.
Phase 1	Smoke test	Complete	Plain chat, reasoning separation, thinking-off, tool parsing and cached-token details worked	The OpenAI-compatible harness and server features were functional.
E0	Baseline; 30 sessions, 152 requests	Complete	1 error; 1,068.5 s; TTFT p50 0.986 s; p99 3.716 s; ITL p50 0.183 s; E2E p50 44.973 s; output 34.814 tok/s; cached 0.808; truncated 0.105	The baseline had good cache reuse but long agent-session latency and context pressure.
E0	Concurrency C1	Complete	32 sessions; 136 requests; 10,150.3 s	Low concurrency avoided contention but underused the GPU.
E0	Concurrency C2	Complete	25 sessions; 109 requests; 1,271.8 s	More concurrency improved utilization.
E0	Concurrency C4	Complete	25 sessions; 109 requests; 729.4 s	Total workload duration fell further.

E0	Concurrency C8	Complete	25 sessions; 109 requests; 627.1 s	Higher concurrency increased aggregate work rate.
E0	Concurrency C16	Complete	25 sessions; 109 requests; 296.6 s	Much higher throughput, with more memory and queue pressure.
E0	Concurrency C32	Complete	20 sessions; 91 requests; 214.7 s	Fastest short run, but not directly comparable because fewer requests completed.
E3	Prefix caching; P4/P8/P16	Complete	Ideal hit rates: 0.818, 0.813, 0.812	Cache reuse declined slightly as project diversity increased.
E4	Batch sizes 512, 2048, 4096, 8192	Complete	All configurations ran	Batch capacity trades throughput against memory use, queueing, and tail latency.
E5	GPU utilization 0.85	Expected failure	Free 5.03 GiB; requested 5.10 GiB	vLLM correctly rejected a configuration without enough startup memory.
E5	GPU utilization 0.80	Complete	5 sessions; 18 requests; 115.2 s	A safer memory setting completed successfully.
E6	N-gram speculative decoding; 3 draft tokens	Complete	Accepted draft length about 1.6-2.4; acceptance about 19.5%-45.5%	Speculation worked, but benefit depended on acceptance rate.
E7	Thinking on/off through replay runner	Not completed	run.py rejected --think	The smoke harness supported the feature, but the full runner did not expose it.
E8	Compiled execution without enforce-eager	Complete	torch.compile took 35.05 s; CUDA graphs captured; graph memory about 0.04 GiB	Compilation adds startup cost and may improve steady-state execution.
E9	NCCL and PCIe inspection	Complete	NCCL world_size=1; backend=nccl; PCIe Gen4 x4	NCCL initialized, but no multi-GPU communication was measured.
E10	CPU weight offload; 5 sessions, 18 requests	Complete	827.1 s; TTFT p50 0.907 s; ITL p50 0.285 s; output 3.614 tok/s	Offloading enabled execution but caused a severe throughput penalty.
E11	FCFS scheduling	Complete	TTFT p50 0.388 s; p99 0.733 s; ITL p50 0.051 s; output 16.746 tok/s	Baseline scheduler comparison.

E11	Priority scheduling	Complete	TTFT p50 0.306 s; p99 0.634 s; ITL p50 0.049 s; output 17.411 tok/s	Slightly better in this small run; per-request priority values were not assigned.
E11	Async scheduling	Complete	TTFT p50 0.234 s; p99 0.573 s; ITL p50 0.037 s; output 22.172 tok/s	Best scheduler result in the measured small workload.
E12	Multiple API-server processes	Unavailable	No api-server-count option in vLLM 0.31.0	The exact experiment could not be performed with the installed version.
E13	Overload; 10 sessions, 45 requests	Complete	0 errors; 412.2 s; TTFT p50 0.660 s; E2E p99 64.281 s; output 25.213 tok/s	The server stayed alive, but tail latency grew.
E13	Long-context test	Complete	4 errors among 5 sessions	Prompt tokens plus reserved output exceeded max-model-len.
E13	Forced OOM; utilization 0.98, context 32K	Expected failure	Free 5.03 GiB; requested 5.88 GiB	Demonstrated vLLM startup memory validation.
E14	Final async configuration; 10 sessions, 45 requests	Complete	0 errors; 581.5 s; TTFT p50 0.593 s; p99 2.305 s; E2E p50 19.567 s; output 18.318 tok/s; cached 0.790; truncated 0.156	Stable final run, but context pressure remained.